

An abstract graphic on the left side of the slide. It features a vertical column of ten glowing blue dots. From each dot, a series of thin, light-blue lines radiate outwards to the right, creating a dense, web-like pattern that resembles a neural network or a stylized brain. The background is a solid dark blue.

第4章 循环神经网络

序列建模



序列数据



新华网 > 财经 > 正文

—2024—

03/21

10:13:55

来源：经济参考报

新一轮工业设备更新或开启四万亿市场

近段时间，多方正加快部署，聚焦钢铁、有色、石化、化工、建材等重点行业，开列项目清单，加大财税金融支持力度，完善用地用能要素保障，推动工业领域大规模设备更新。业内研究预计，随着政策利好释放，我国工业领域设备更新年规模约在4万亿元左右。

接受《经济参考报》记者采访的多位专家表示，工业既是各类设备的供给方，也是设备的需求方。推动工业设备向高端、智能、绿色、安全方向更新升级，将进一步拉动有效投资、提升发展质效，以设备升级带动我国制造业整体竞争力提升。

工业领域设备更新潜力足

日前，国务院印发《推动大规模设备更新和消费品以旧换新行动方案》（以下简称《行动方案》），明确“聚焦钢铁、有色、石化、化工、建材、电力、机械、航空、船舶、轻纺、电子等重点行业，大力推动生产



- 在时间 t 观察到 x_t , 那么得到 T 个不独立的随机变量:

$$(x_1, \dots x_T) \sim p(\mathbf{x})$$

- 使用条件概率展开:

$$p(\mathbf{x}) = p(x_1) \cdot p(x_2 | x_1) \cdot p(x_3 | x_1, x_2) \cdot \dots p(x_T | x_1, \dots x_{T-1})$$

$$p(\mathbf{x}) = p(x_T) \cdot p(x_{T-1} | x_T) \cdot p(x_{T-2} | x_{T-1}, x_T) \cdot \dots p(x_1 | x_2, \dots x_T)$$

- 方案1：马尔科夫假设

- 假设当前数据只跟过去 τ 个数据点有关

$$p(x_t | x_1, \dots, x_{t-1}) = p(x_t | x_{t-\tau}, \dots, x_{t-1}) = p(x_t | f(x_{t-\tau}, \dots, x_{t-1}))$$

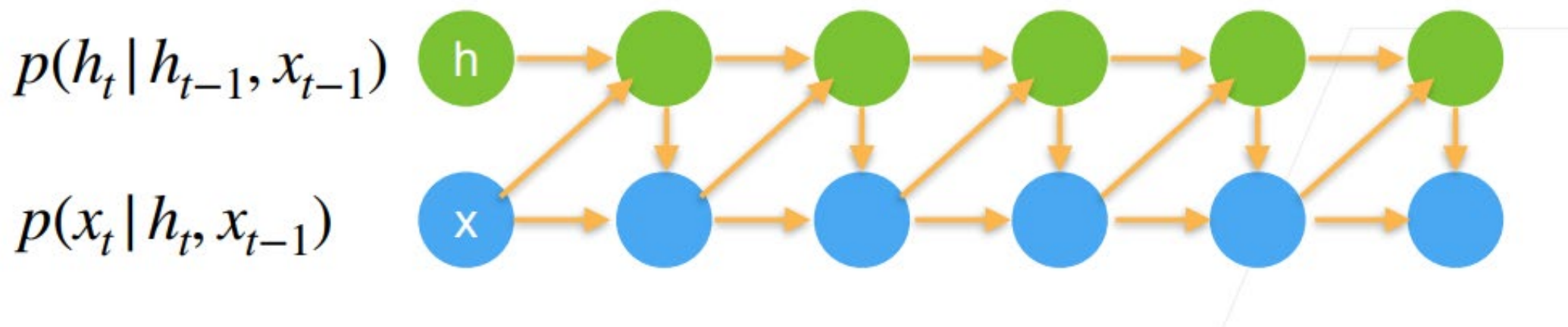
在过去 τ 个数据上建模，例如在过去的
数据上建立一个MLP模型

- 方案2：潜变量模型 (Latent Variable Model)

- 引入潜变量 h_t 来表示过去的信息： $h_t = f(x_1, \dots, x_{t-1})$

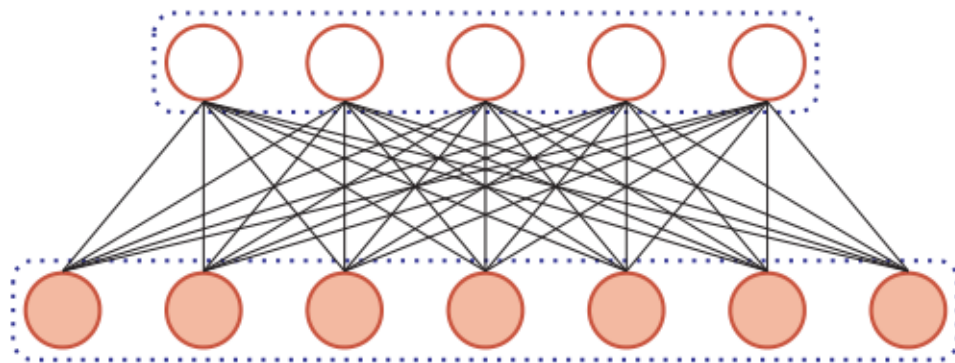
这样： $x_t = p(x_t | h_t)$

- 潜变量自回归模型：使用潜变量 h_t 总结过去的信息

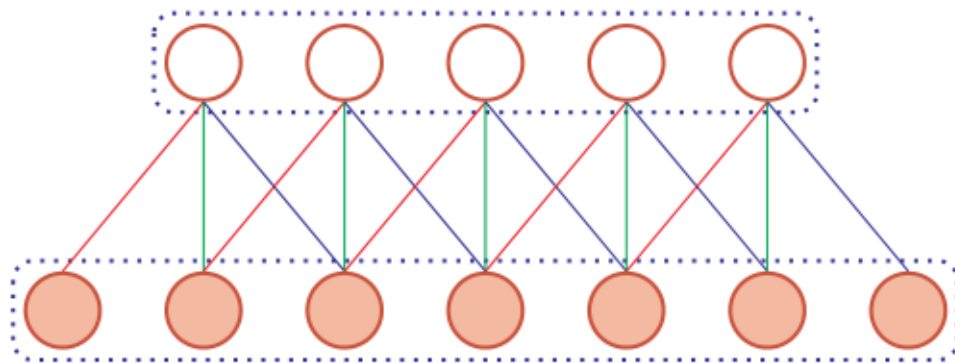


前馈神经网络能否用于序列建模？

- 前馈网络的连接存在层与层之间，每层的节点之间是无连接的。（无记忆）
- 输入和输出的维数都是固定的，不能任意改变。（无法处理变长的序列数据）



(a) 全连接层



(b) 卷积层

循环神经网络

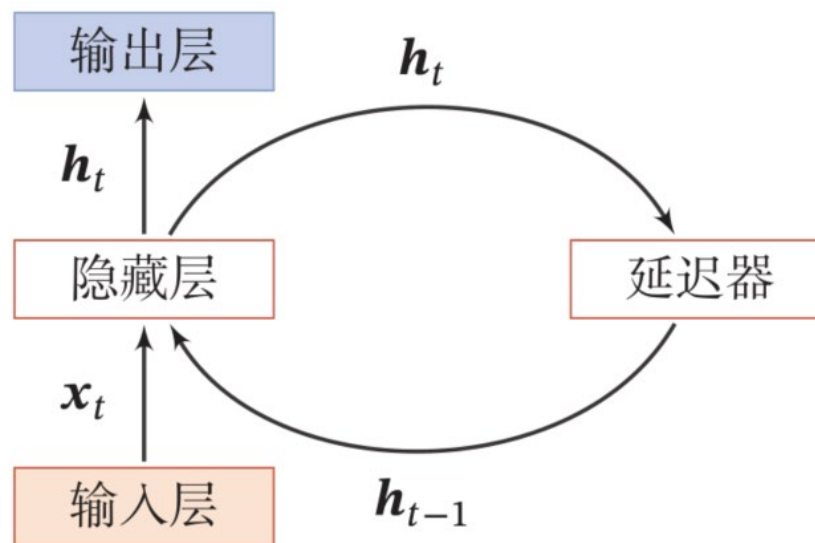


循环神经网络（Recurrent Neural Network，RNN）

- 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

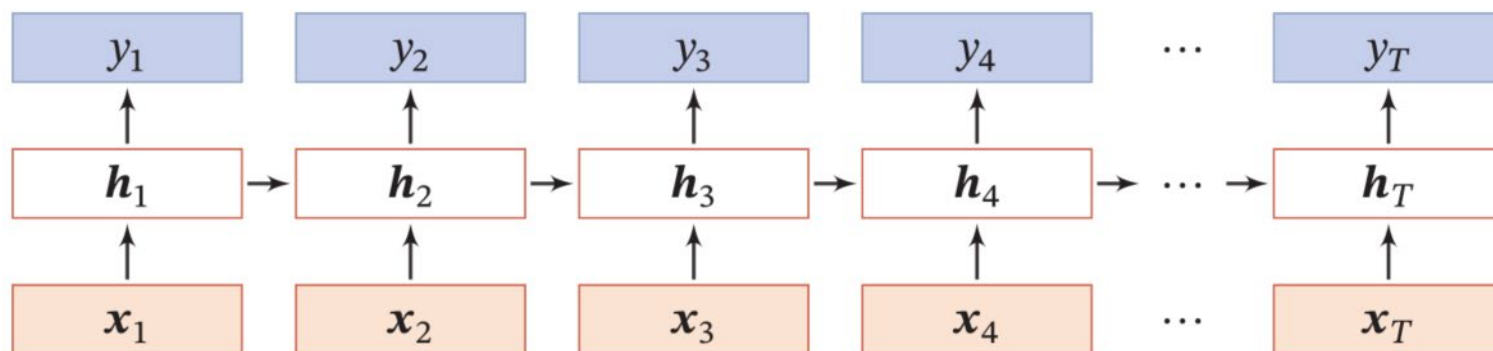
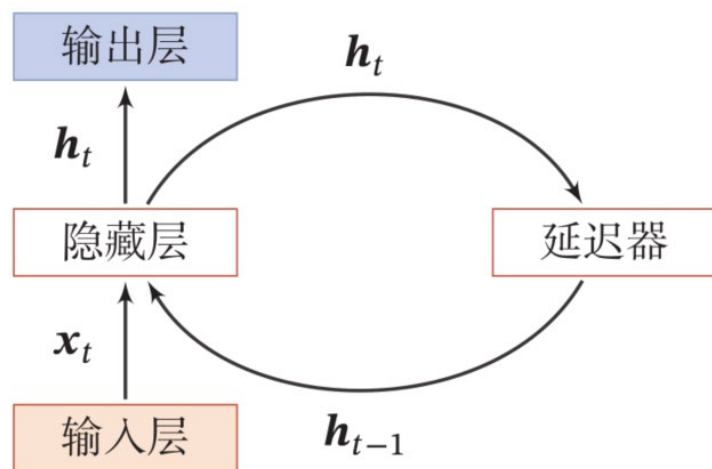
$$h_t = f(h_{t-1}, x_t)$$

活性值
状态

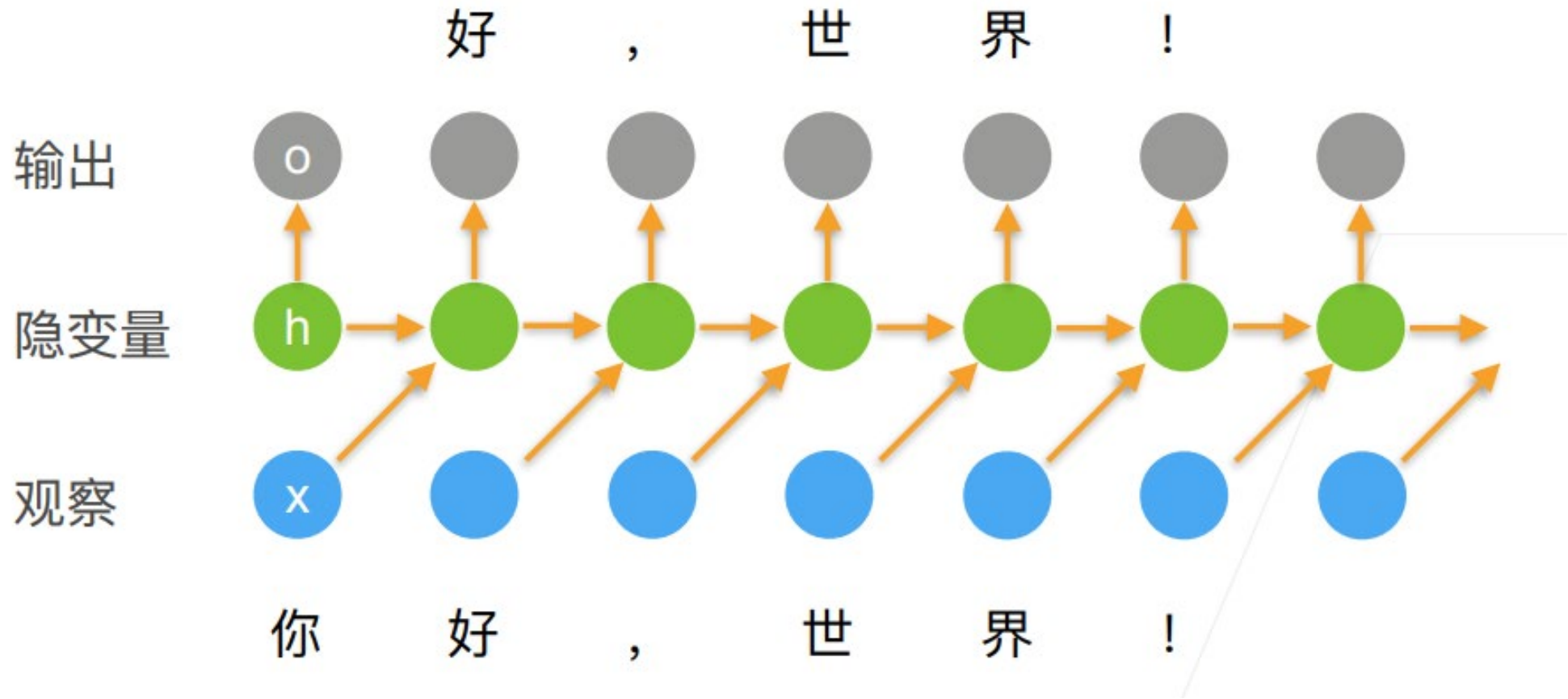


- 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- 循环神经网络已经被广泛应用于语音识别、语言模型以及自然语言生成等任务上

按时间展开



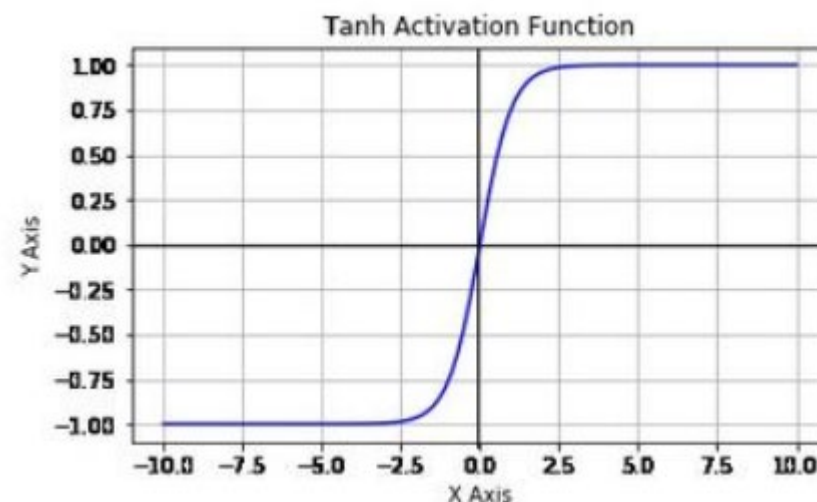
简单循环网络 (Simple Recurrent Network , SRN)



循环神经网络中的激活函数

- RNN通常可使用tanh激活函数：

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$



- 也可以使用ReLU激活函数（但是不良的参数初始化容易导致梯度爆炸）。

简单循环网络

- 状态更新:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

- 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元, 它可以以任意的准确率去近似任何一个非线性动力系统

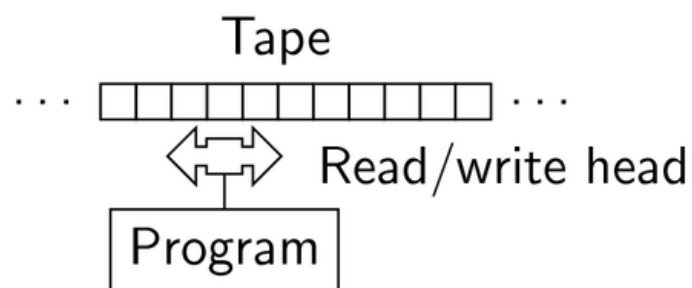
$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态, $\mathbf{x}_t$ 是外部输入, $g(\cdot)$ 是可测的状态转换函数, $o(\cdot)$ 是连续输出函数, 并且对状态空间的紧致性没有限制.

图灵完备

- 图灵完备 (Turing Completeness) 是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。



定理 6.2 – 图灵完备 [Siegelmann et al., 1991]: 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟.

- 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

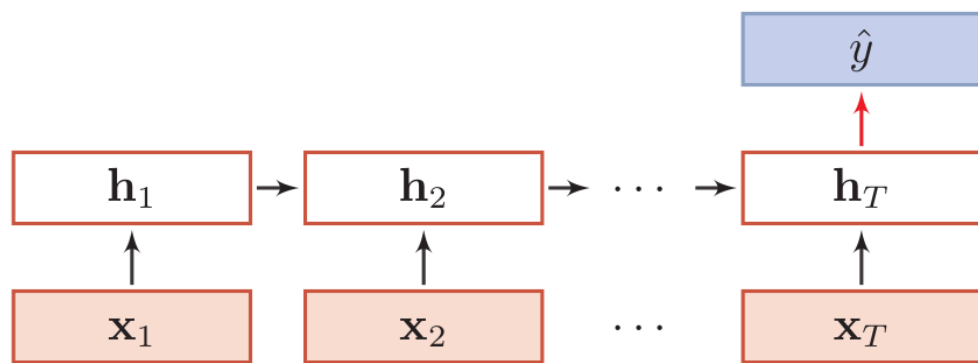
循环神经网络应用到机器学习



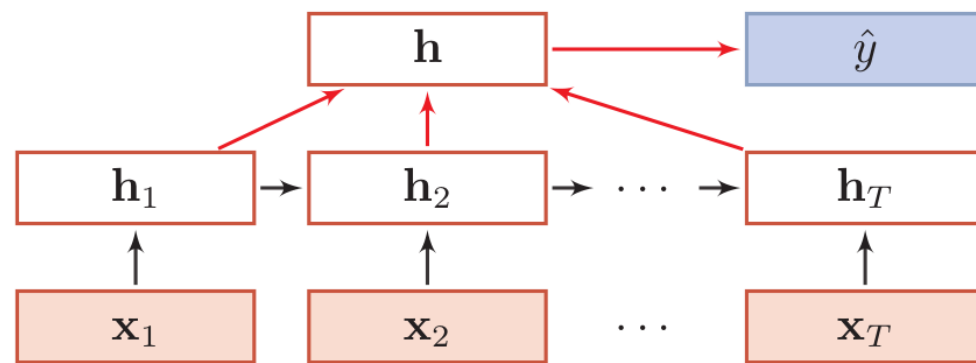
应用到机器学习

- **情况1：序列到类别**
- **情况2：同步的序列到序列模式**
- **情况3：异步的序列到序列模式**

序列到类别



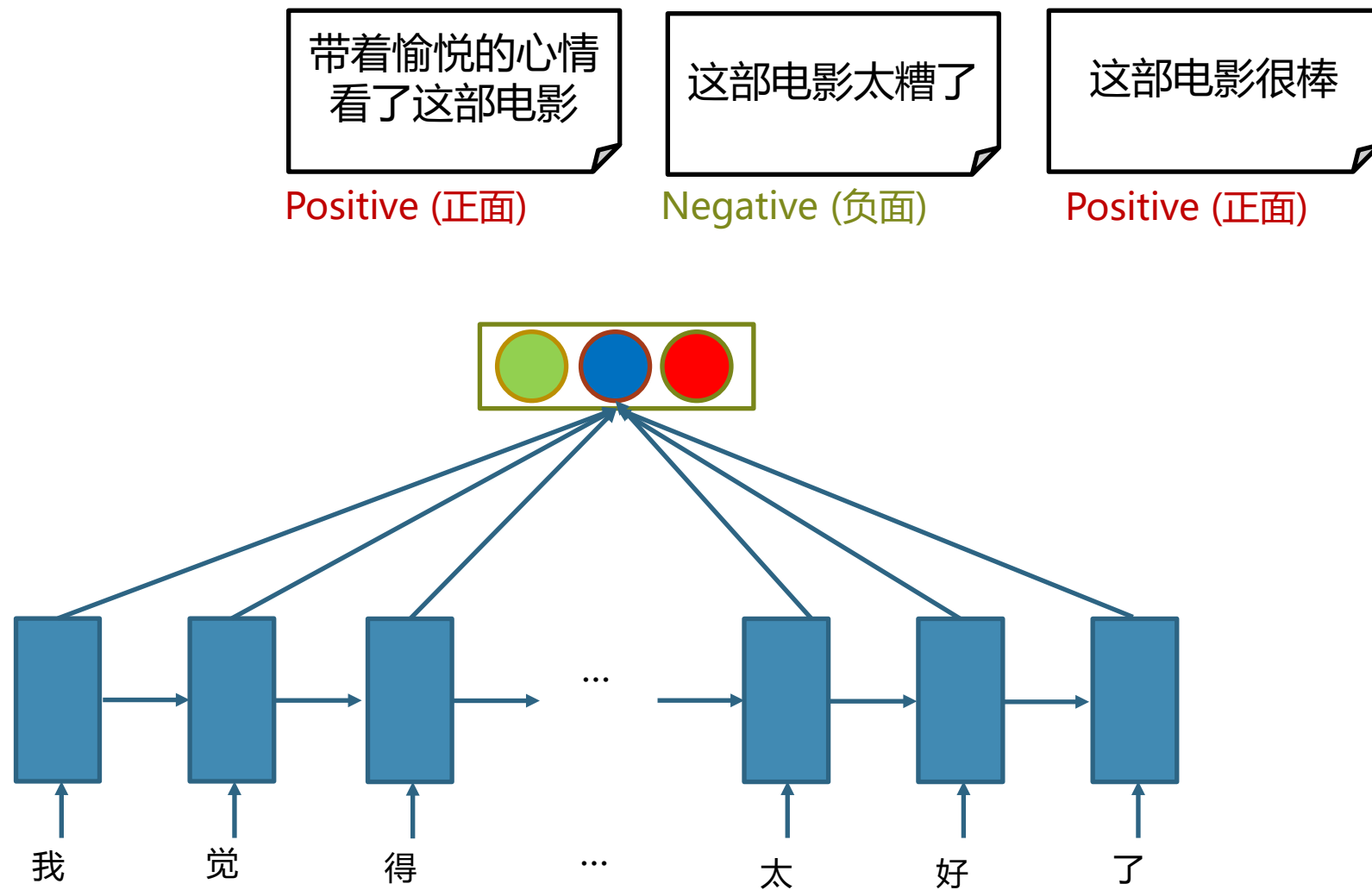
(a) 正常模式



(b) 按时间进行平均采样模式

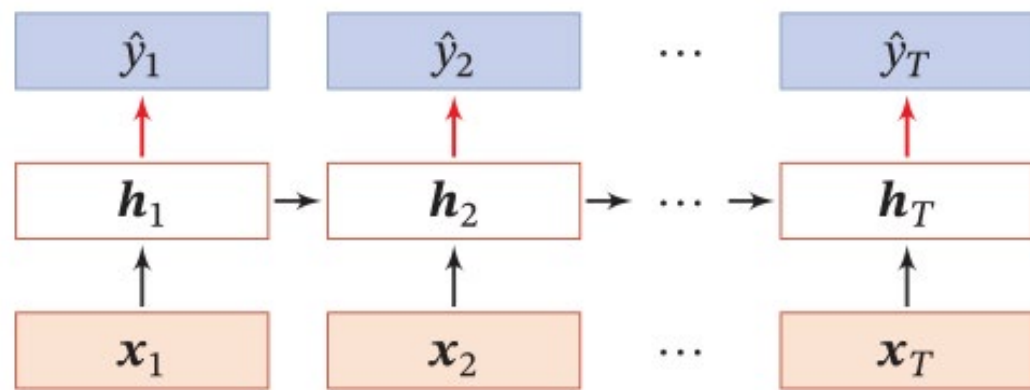
序列到类别

- 情感分类



同步的序列到序列

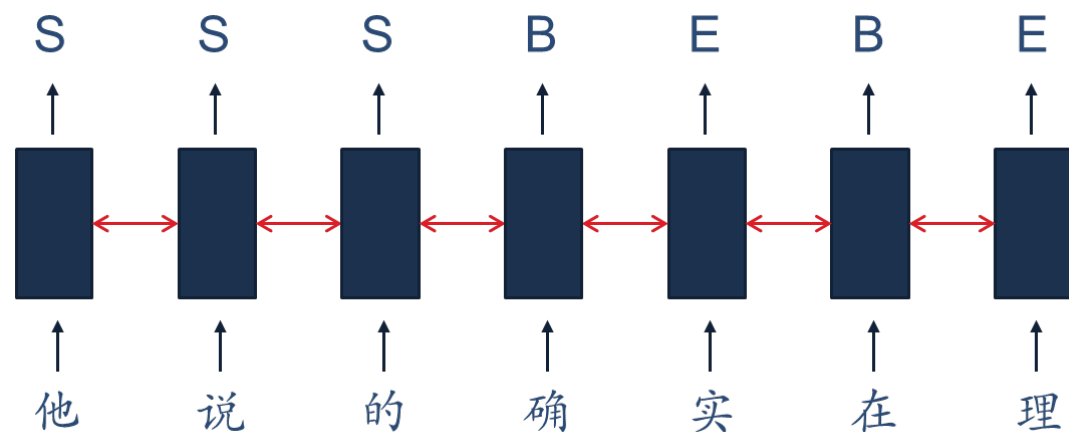
- 同步的序列到序列模式



- 中文分词

- 英文词性标注

- Token分析

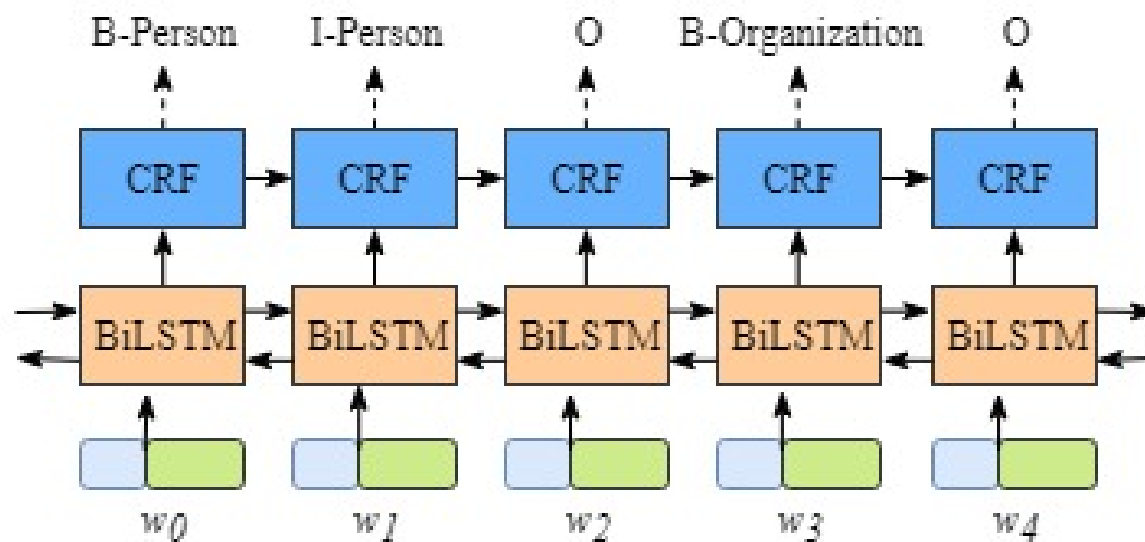


同步的序列到序列

- 信息抽取(Information Extraction, IE)

- 从无结构的文本中抽取结构化的信息，形成知识

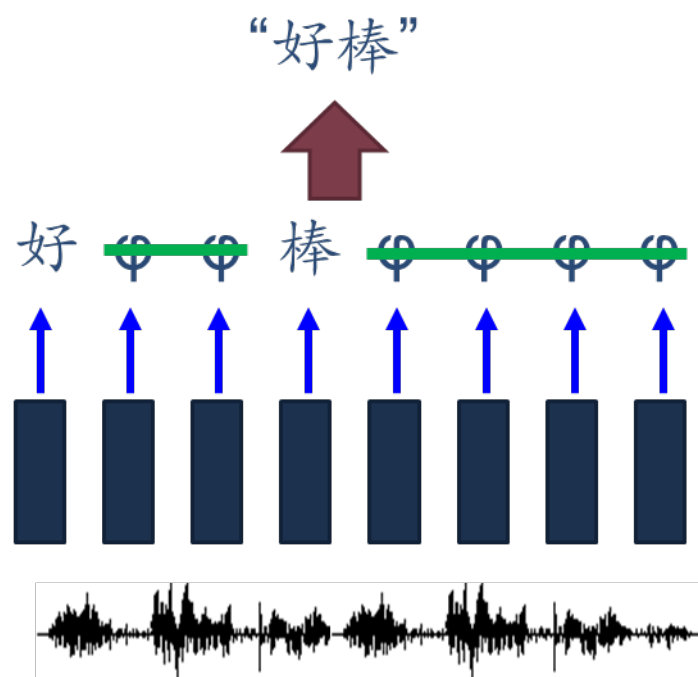
小米创始人雷军表示，该公司2015年营收达到780亿元人民币，较2014年的743亿元人民币增长了5%。



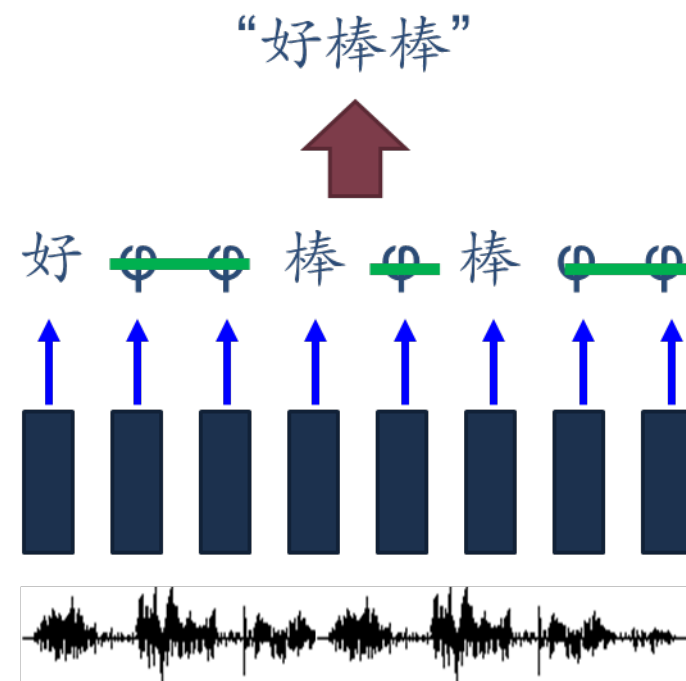
同步的序列到序列

● 语音识别——连接时序分类

- Connectionist Temporal Classification (CTC) [Alex Graves, ICML' 06][Alex Graves, ICML' 14][Haşim Sak, Interspeech' 15][Jie Li, Interspeech' 15][Andrew Senior, ASRU' 15]

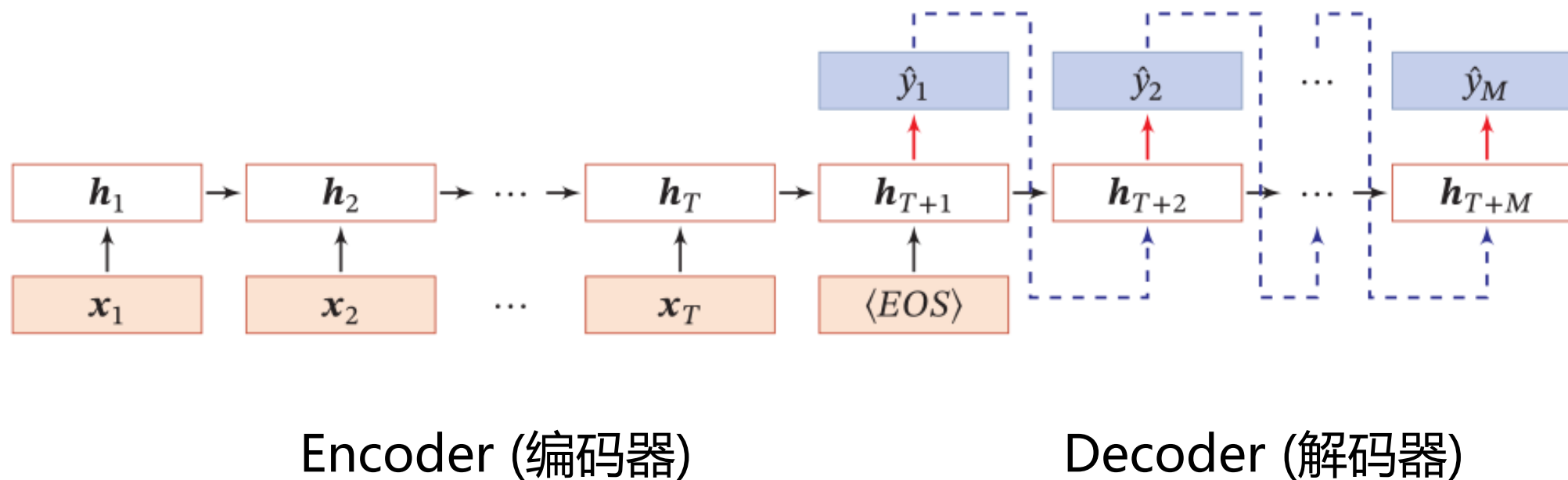


语音识别



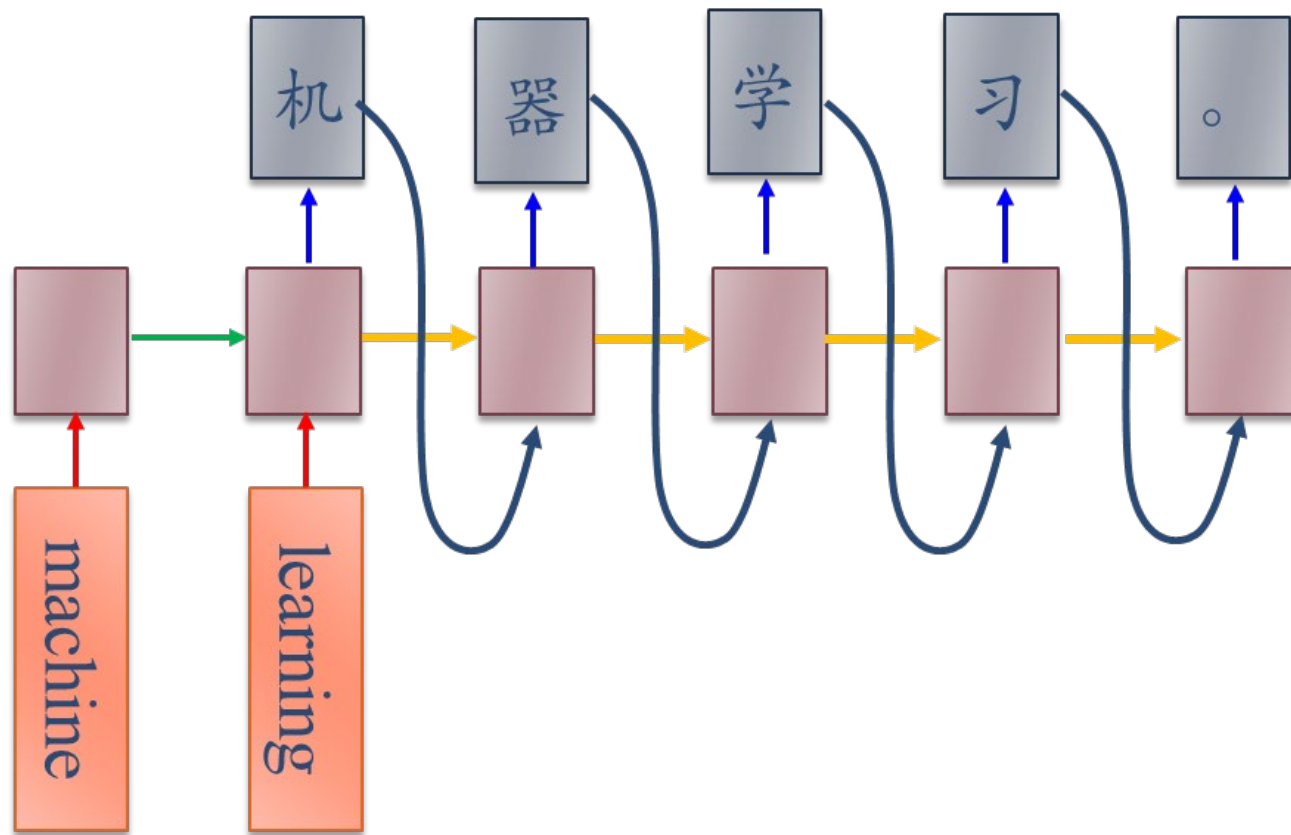
异步的序列到序列

- 异步的序列到序列模式



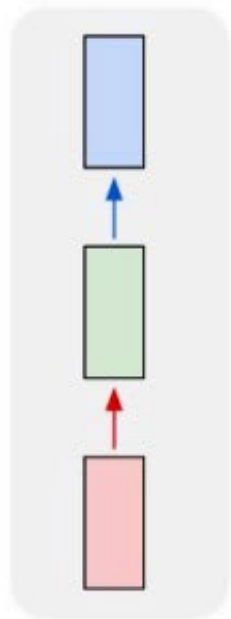
异步的序列到序列

- 机器翻译

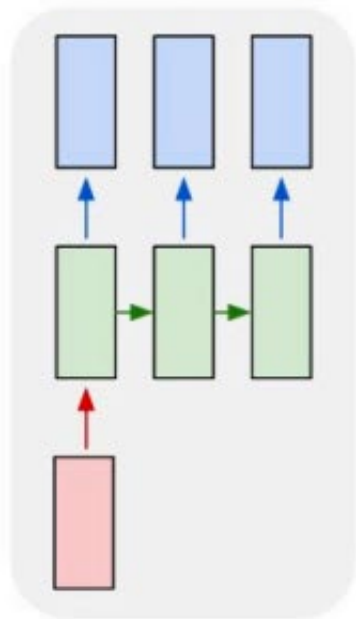


不同模式的总结

one to one

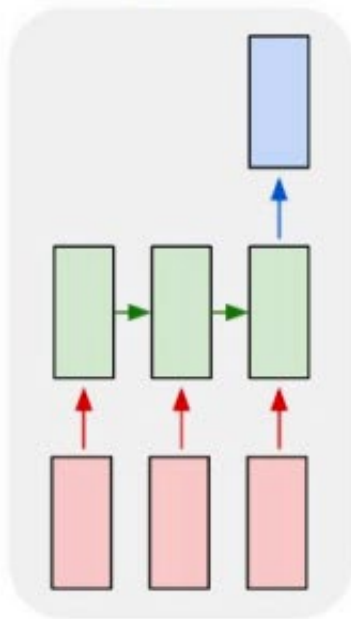


one to many



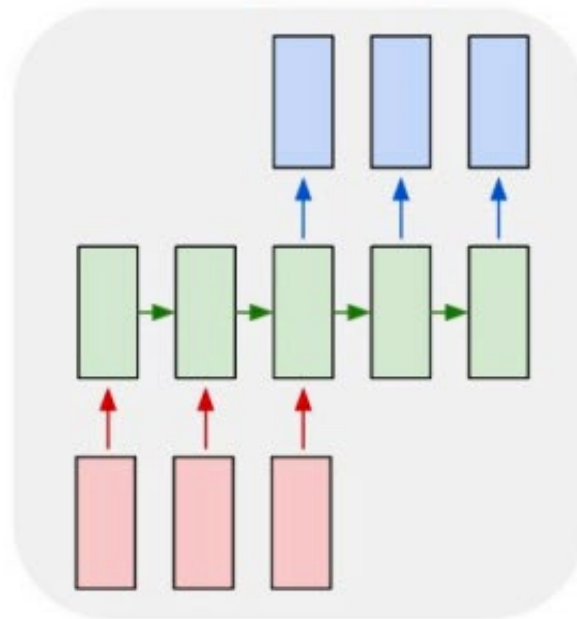
文本生成

many to one



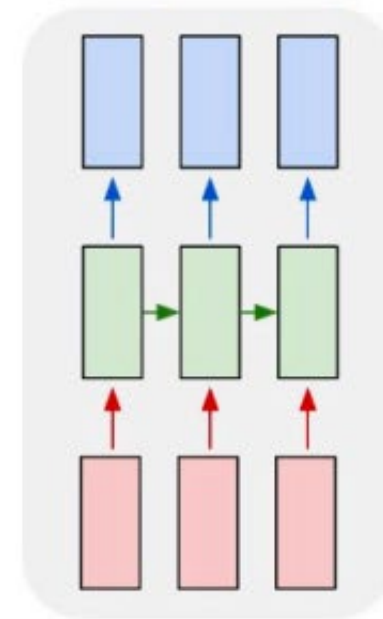
文本分类

many to many



问答、机器翻译

many to many



Tag生成

参数学习



● 机器学习

- 给定一个训练样本 (x, y) , 其中

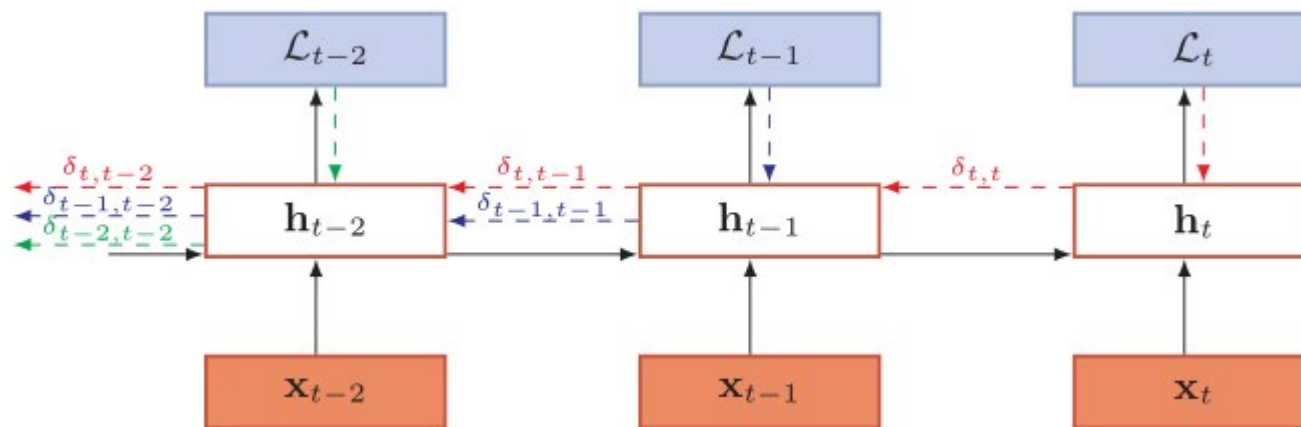
- $x = (x_1, \dots, x_T)$ 为长度是 T 的输入序列,
- $y = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

- 时刻 t 的瞬时损失函数为 $\mathcal{L}_t = \mathcal{L}(y_t, g(\mathbf{h}_t))$,

- 总损失函数 $\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t$.

- 随时间反向传播算法 (BPTT)

$$\mathbf{h}_{t+1} = f(\mathbf{z}_{t+1}) = f(U\mathbf{h}_t + W\mathbf{x}_{t+1} + \mathbf{b})$$



$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(\mathbf{z}_{\tau})) U^T \right) \delta_{t,t}$$

$\delta_{t,k}$ 为第 t 时刻的损失对第 k 步隐藏神经元的净输入 $\mathbf{z}_k$ 的导数

梯度消失、梯度爆炸

- 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

- 其中

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \underbrace{\left(\text{diag}(f'(\mathbf{z}_{\tau})) U^T \right)}_{\gamma} \delta_{t,t}$$

$$\delta_{t,k} \cong \gamma^{t-k} \delta_{t,t}$$

由于梯度爆炸或消失问题，RNN实际上只能学习到短周期的依赖关系。这就是所谓的**长程依赖问题**。

长程依赖问题

- 循环神经网络在时间维度上非常深!

- 梯度消失或梯度爆炸

- 如何改进?

- 梯度爆炸问题

- 权重衰减——如添加正则化项
- 梯度截断——设置梯度上限 θ

$$\mathbf{g} \leftarrow \min \left(1, \frac{\theta}{\|\mathbf{g}\|} \right) \mathbf{g}$$

- 梯度消失问题

- 改进模型

GRU和LSTM



门控机制

- 长程依赖问题

- 门控机制

- 控制信息的累积速度，包括有选择地加入新的信息，以及有选择地遗忘之前累积的信息。

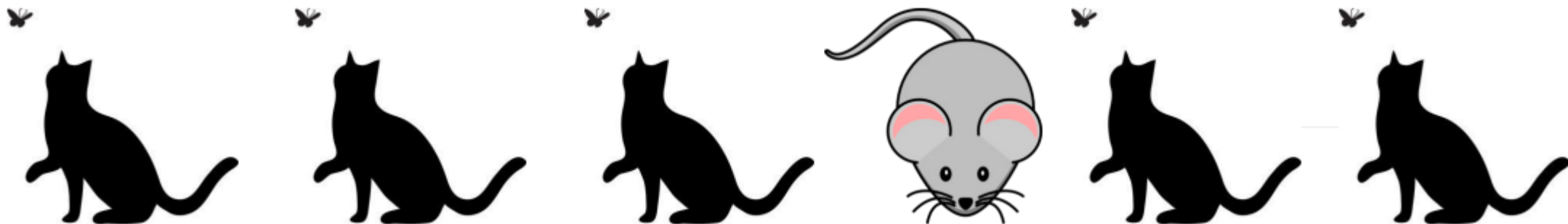
- 基于门控的循环神经网络 (Gated RNN)

- 门控循环单元 (GRU)
- 长短期记忆神经网络 (LSTM)



门控循环单元 (GRU)

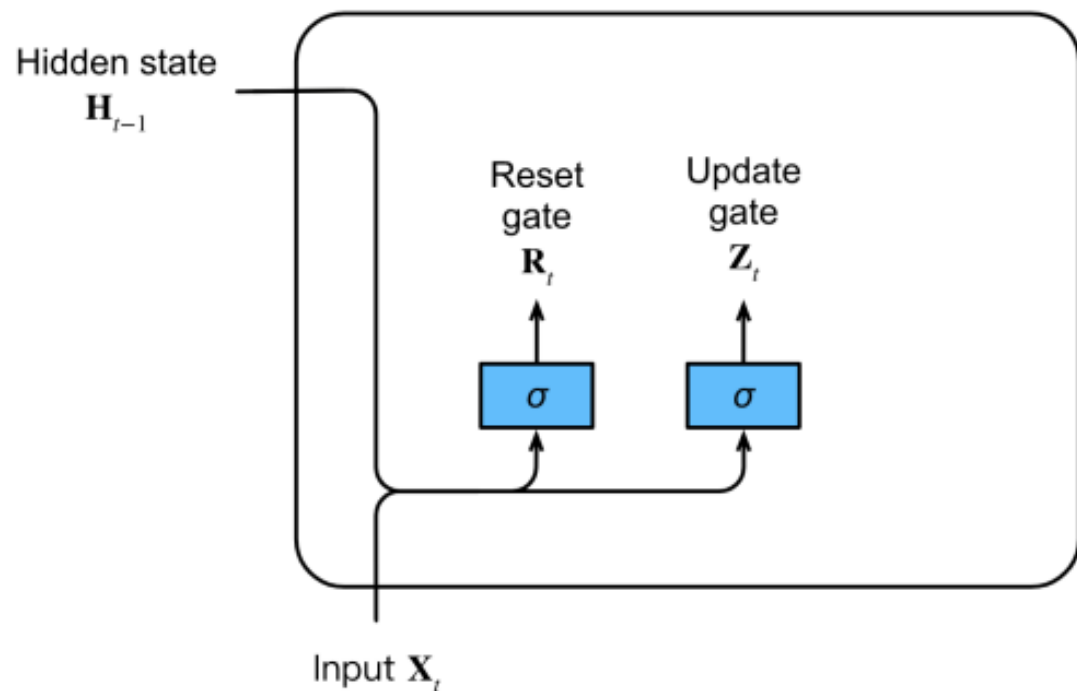
- 不是每个观测值都同等重要



- 只需要记住重要观测:

- 能关注的机制 (更新门)
- 能遗忘的机制 (重置门)

门控循环单元 (GRU)

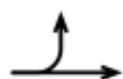


重置门 $R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$

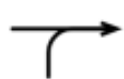
更新门 $Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$



FC layer with
activation function

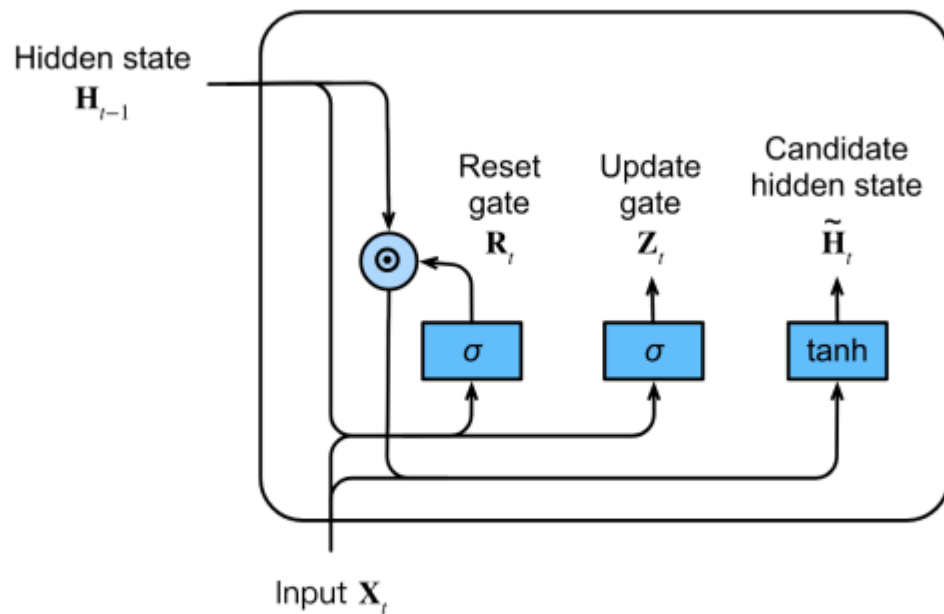


Copy



Concatenate

门控循环单元 (GRU)



候选隐状态

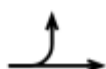
$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$



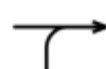
FC layer with
activation function



Elementwise
operator

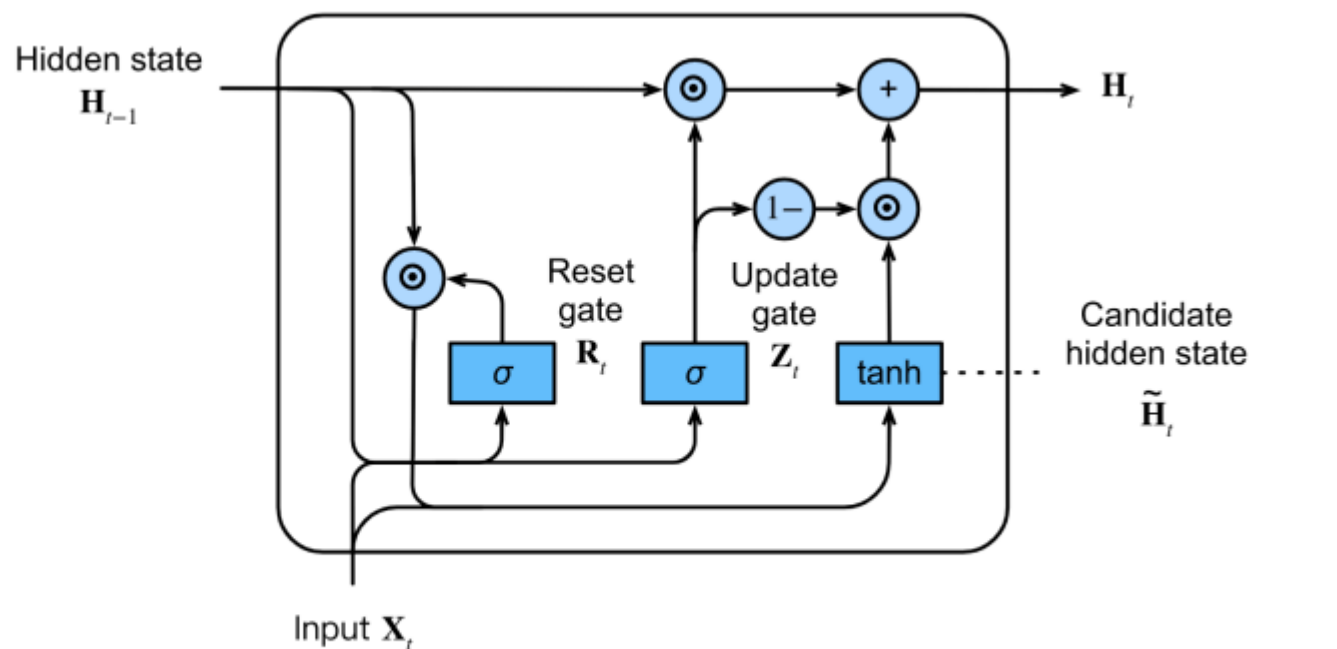


Copy



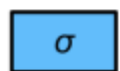
Concatenate

门控循环单元 (GRU)



隐状态

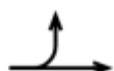
$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$



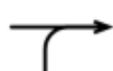
FC layer with
activation fuction



Elementwise
operator

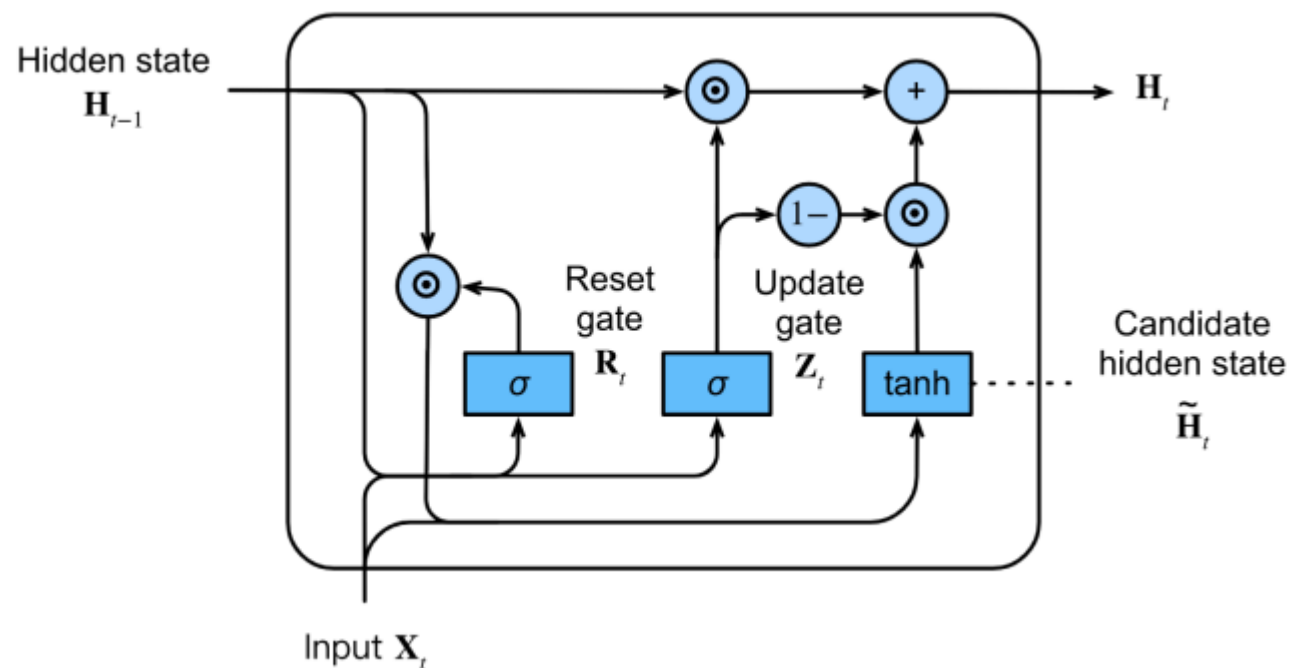


Copy



Concatenate

门控循环单元 (GRU)



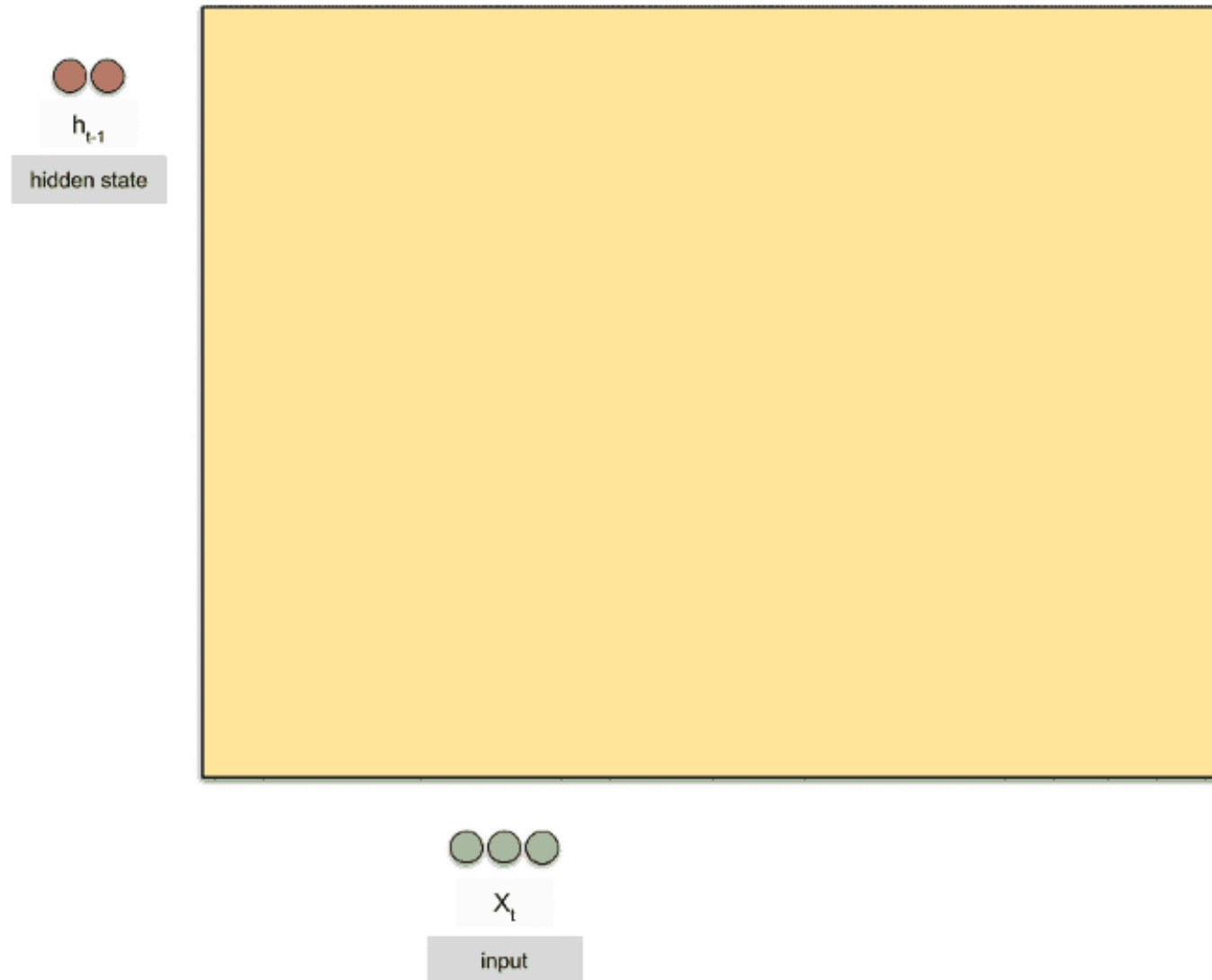
$$R_t = \sigma(X_t W_{xr} + H_{t-1} W_{hr} + b_r),$$

$$Z_t = \sigma(X_t W_{xz} + H_{t-1} W_{hz} + b_z)$$

$$\tilde{H}_t = \tanh(X_t W_{xh} + (R_t \odot H_{t-1}) W_{hh} + b_h)$$

$$H_t = Z_t \odot H_{t-1} + (1 - Z_t) \odot \tilde{H}_t$$

门控循环单元 (GRU)



长短期记忆神经网络 (LSTM)

- 遗忘门：将传递的信息减少
- 输入门：决定以多大比例忽略输入数据
- 输出门：决定以多大比例使用隐状态

LONG SHORT-TERM MEMORY

NEURAL COMPUTATION 9(8):1735–1780, 1997

Sepp Hochreiter

Fakultät für Informatik

Technische Universität München

80290 München, Germany

hochreit@informatik.tu-muenchen.de

<http://www7.informatik.tu-muenchen.de/~hochreit>

Jürgen Schmidhuber

IDSIA

Corso Elvezia 36

6900 Lugano, Switzerland

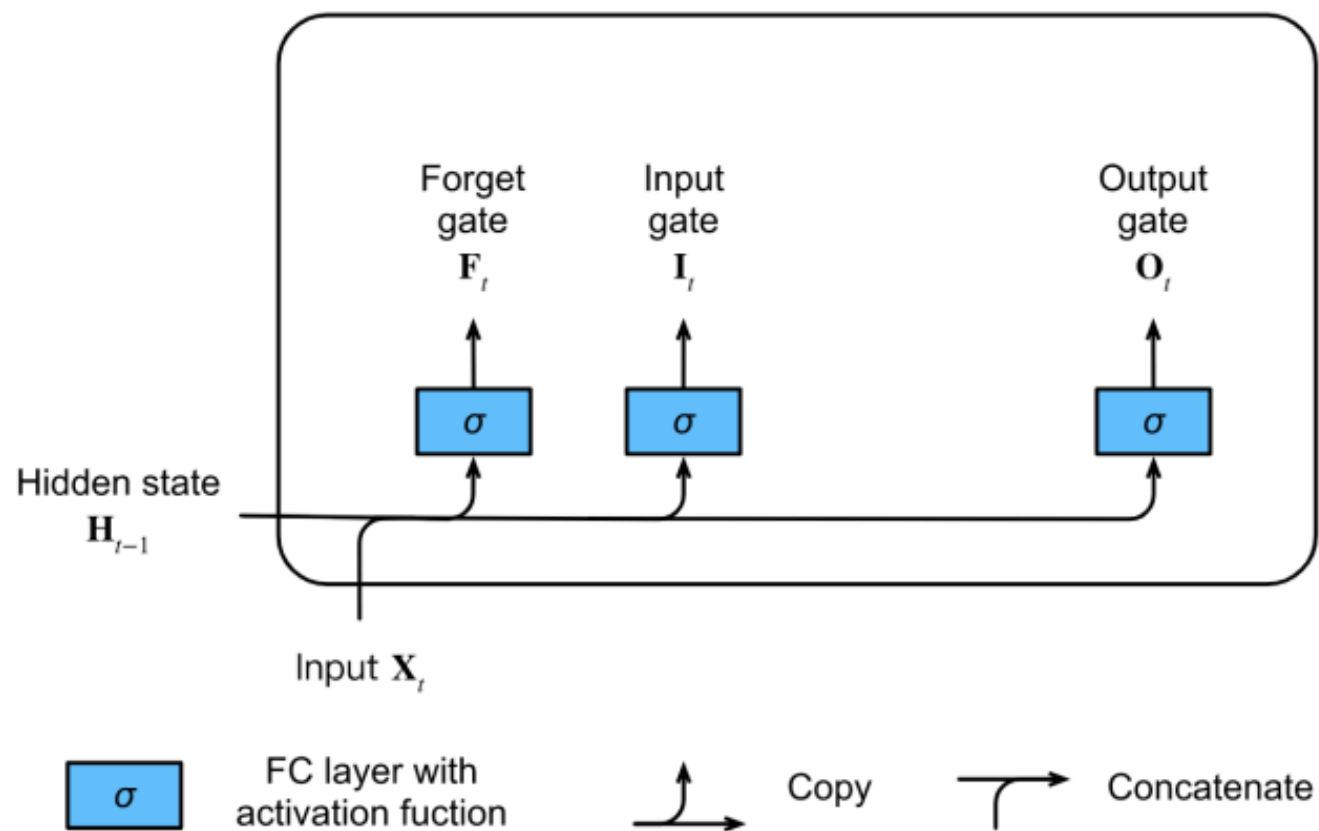
juergen@idsia.ch

<http://www.idsia.ch/~juergen>



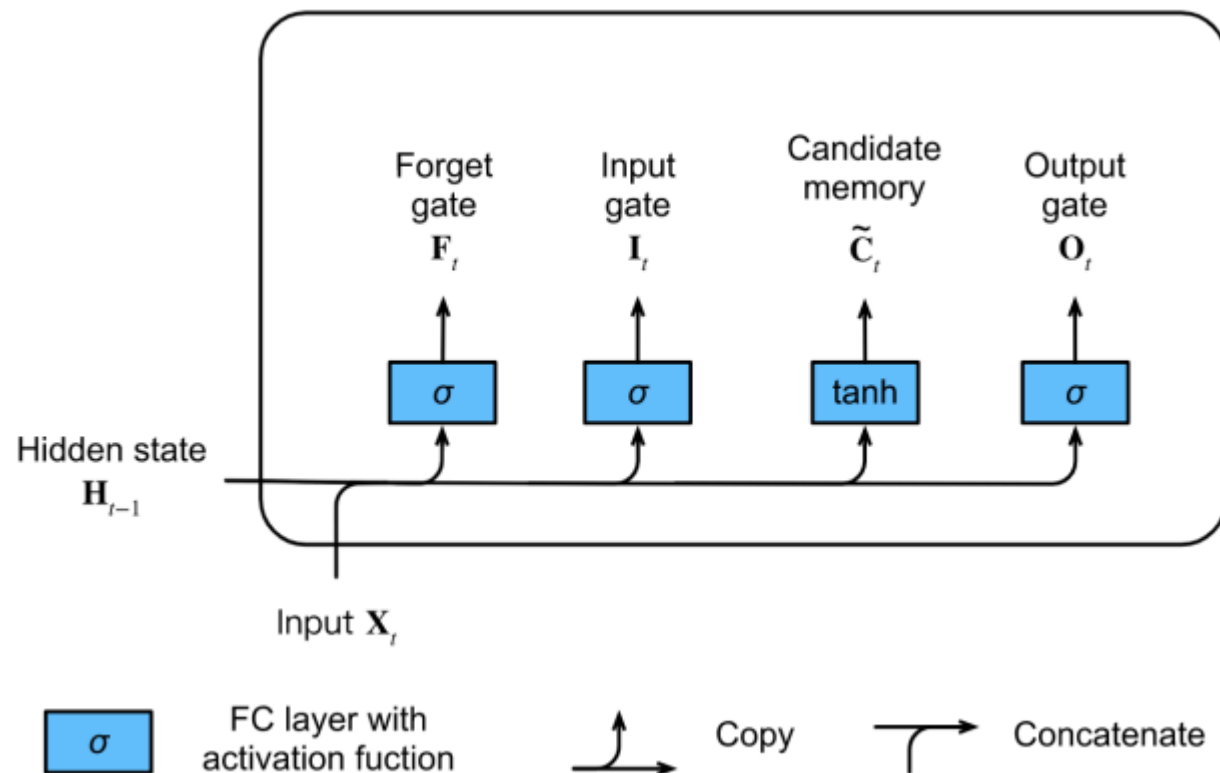
Jürgen Schmidhuber

长短期记忆神经网络 (LSTM)



$$I_t = \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i)$$
$$F_t = \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f)$$
$$O_t = \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o)$$

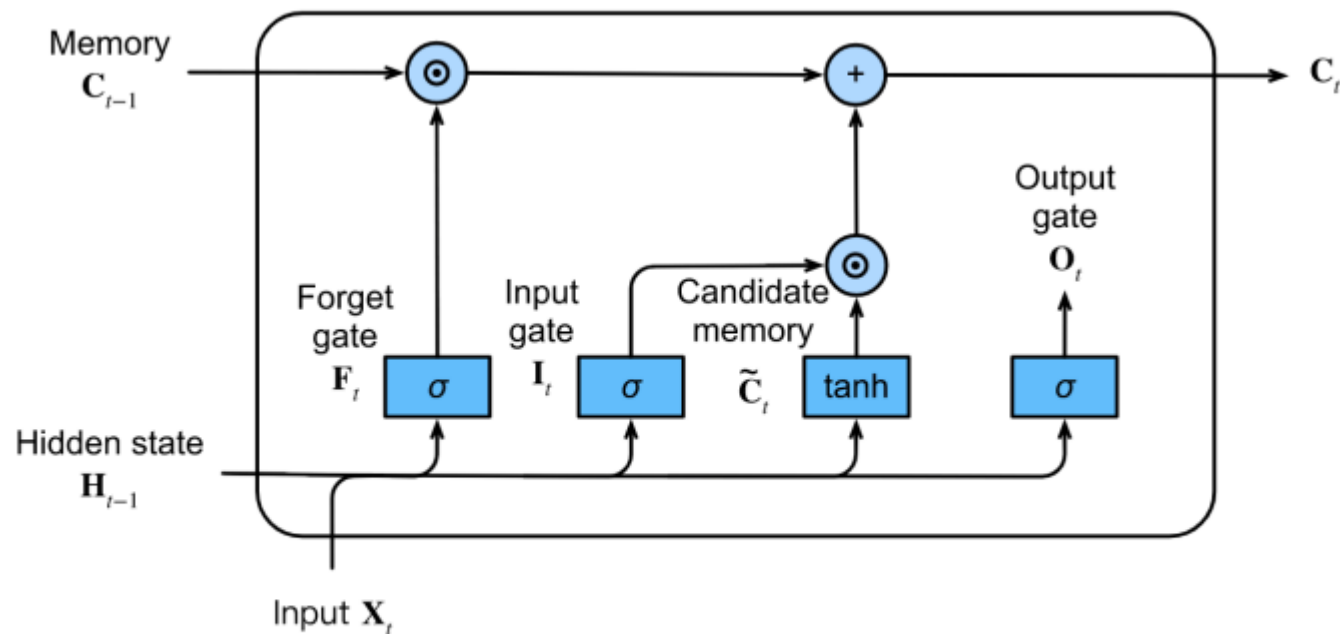
长短期记忆神经网络 (LSTM)



候选记忆单元

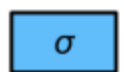
$$\tilde{C}_t = \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c)$$

长短期记忆神经网络 (LSTM)



记忆单元

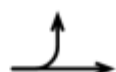
$$C_t = F_t \odot C_{t-1} + I_t \odot \tilde{C}_t$$



FC layer with
activation function



Elementwise
operator

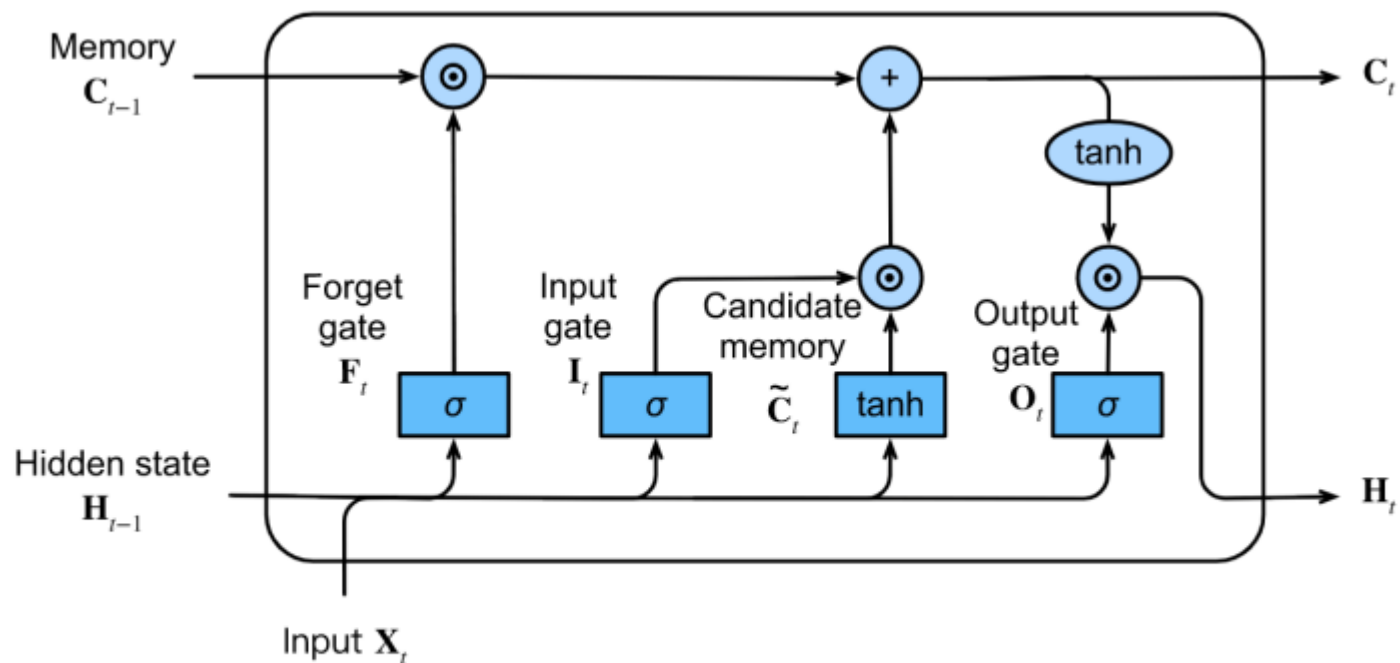


Copy



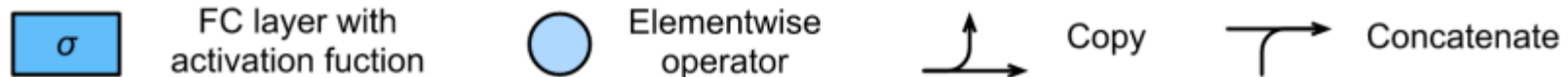
Concatenate

长短期记忆神经网络 (LSTM)

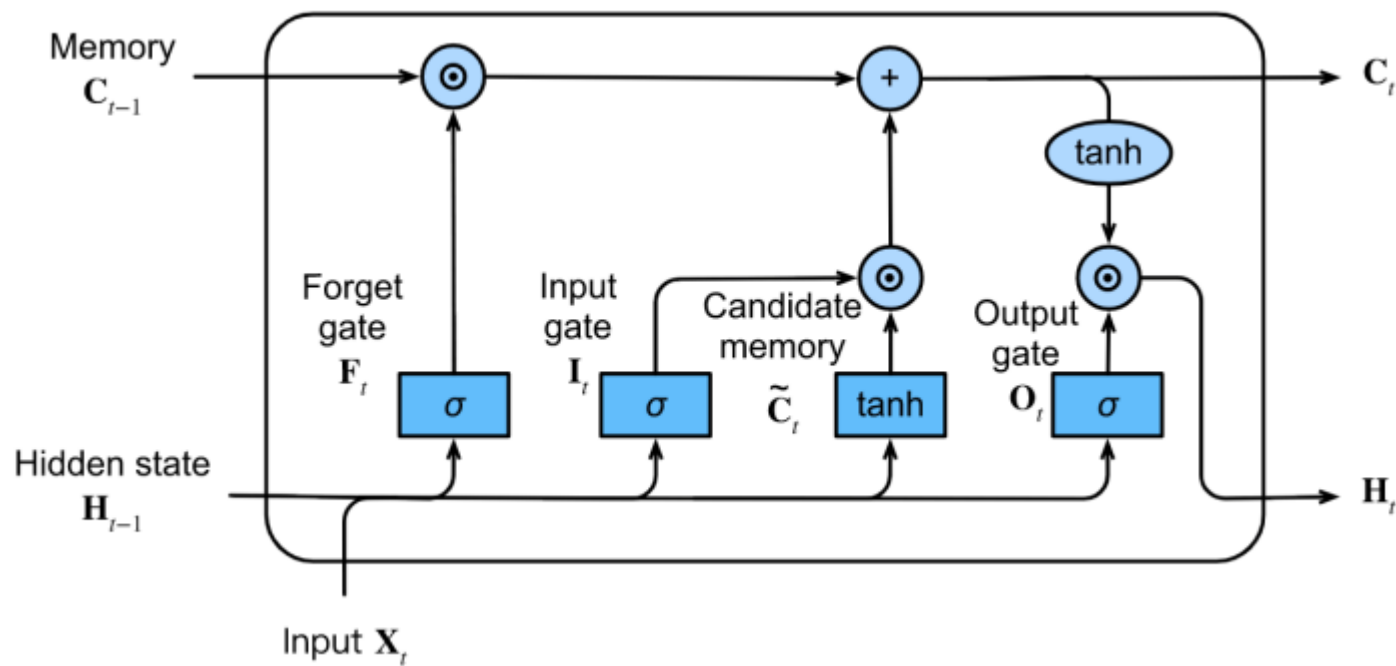


隐状态

$$H_t = O_t \odot \tanh(C_t)$$

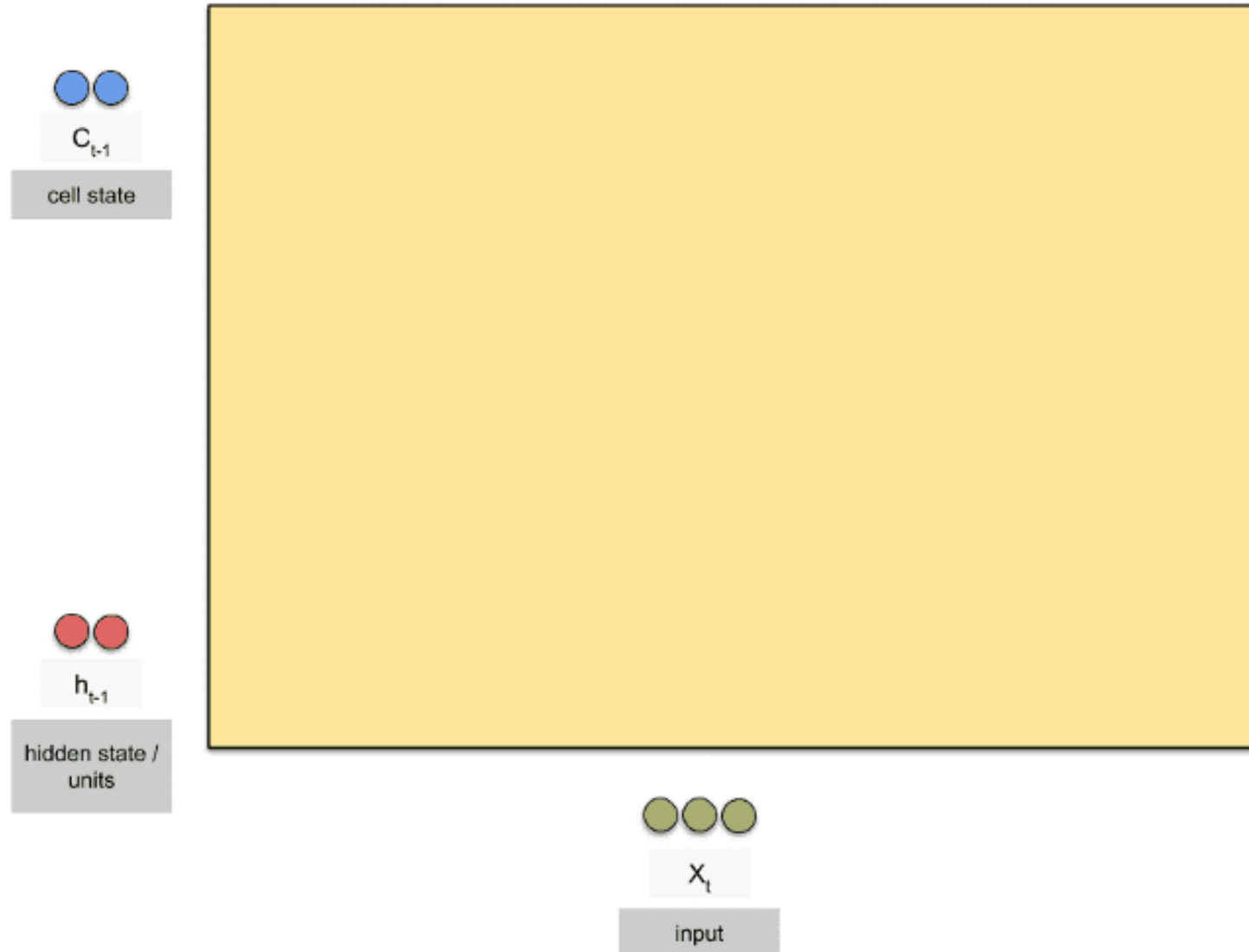


长短期记忆神经网络 (LSTM)



$$\begin{aligned} I_t &= \sigma(X_t W_{xi} + H_{t-1} W_{hi} + b_i) \\ F_t &= \sigma(X_t W_{xf} + H_{t-1} W_{hf} + b_f) \\ O_t &= \sigma(X_t W_{xo} + H_{t-1} W_{ho} + b_o) \\ \tilde{C}_t &= \tanh(X_t W_{xc} + H_{t-1} W_{hc} + b_c) \\ C_t &= F_t \odot C_{t-1} + I_t \odot \tilde{C}_t \\ H_t &= O_t \odot \tanh(C_t) \end{aligned}$$

长短期记忆神经网络 (LSTM)



LSTM的各种变体

- 没有遗忘门（早期的LSTM）

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t.$$

- 耦合输入门和遗忘门

$$\mathbf{f}_t + \mathbf{i}_t = \mathbf{1}.$$

$$\mathbf{c}_t = (\mathbf{1} - \mathbf{i}_t) \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

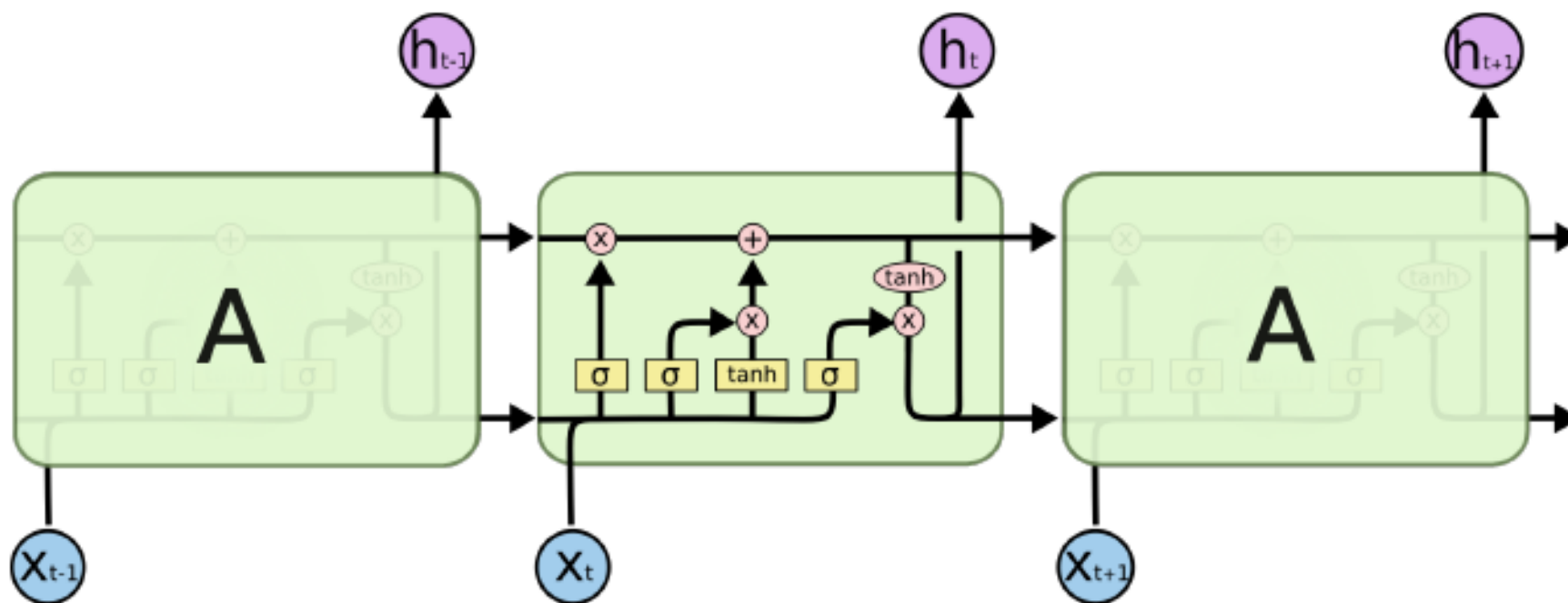
- peephole连接

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + V_i \mathbf{c}_{t-1} + \mathbf{b}_i),$$

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + V_f \mathbf{c}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + V_o \mathbf{c}_t + \mathbf{b}_o),$$

- 观察模型架构，并回答以下问题：



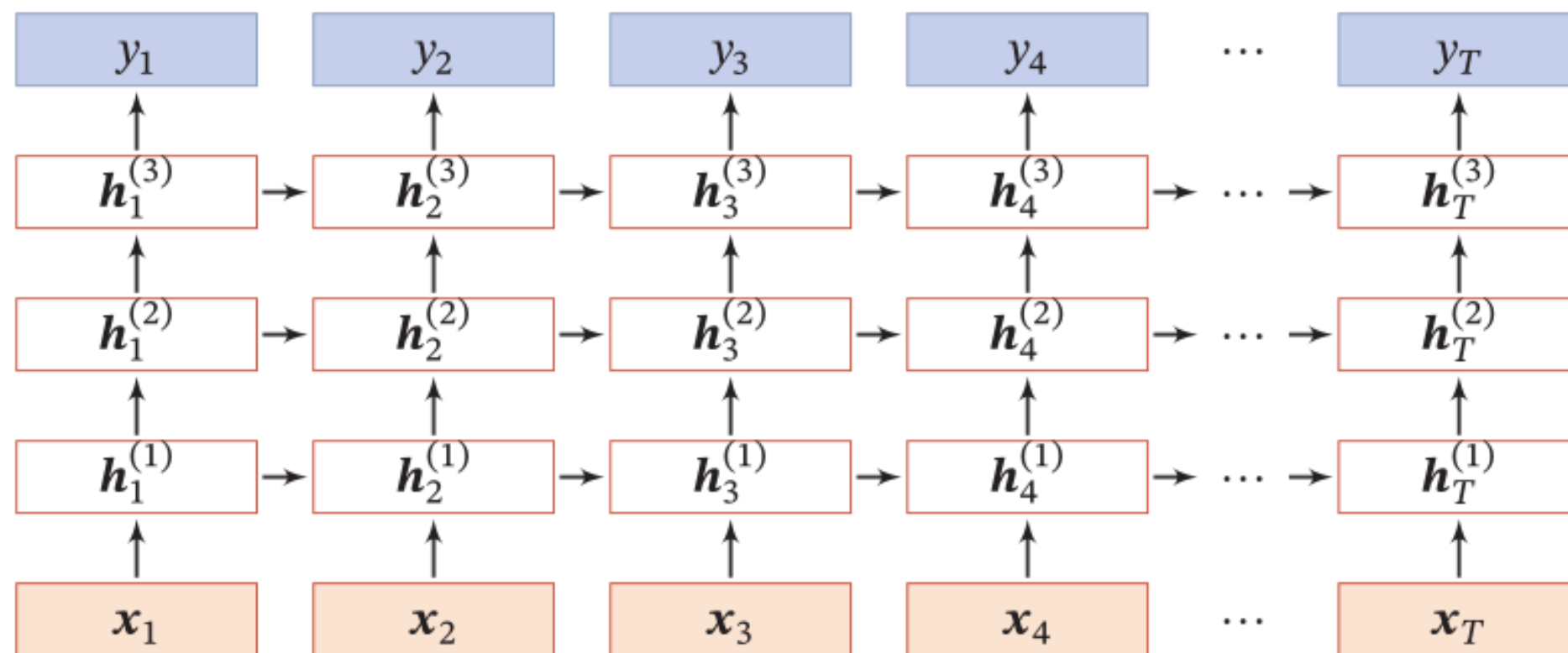
- 请写出图中所示模型的名称，并简要陈述你的判断依据；
- 图中所示模型使用了哪些门控单元？试标注在图上，并简述这些门控单元的作用。

- 对于RNN而言，同步的序列到序列模式有哪些应用场景？异步的序列到序列模式有哪些应用场景？
- RNN模型存在的主要问题是什么？如何解决这一问题？
- 判断正误：
 - 与卷积神经网络相比，循环神经网络同样具有局部连接和权值共享的特点；
 - 循环神经网络在整个序列的损失可定义为不同时间步的损失之和；
 - 循环神经网络可通过随时间的反向传播算法（BPTT）进行训练，由于不同时刻的状态相互依赖，所以需要存储每个时刻的状态信息，难以并行计算；
 - 在GRU中，重置门决定先前隐藏状态单元是否被忽略，而更新门控制当前隐藏状态单元是否需要被新的隐藏状态单元更新；
 - 在GRU中，增加了一个记忆单元 C_t ，其在不同时刻有着可变的连接权重，以缓解梯度消失和梯度爆炸问题。

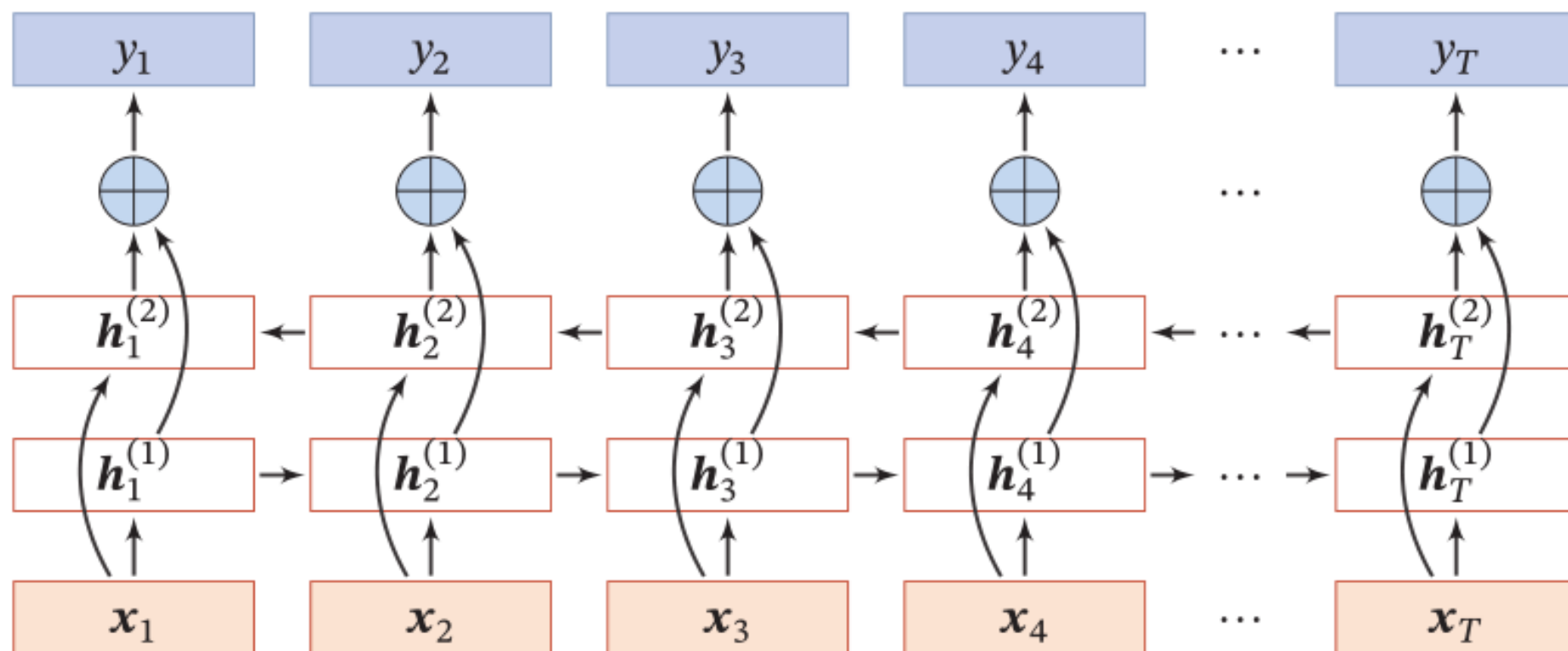
深层循环神经网络



堆叠循环神经网络



双向循环神经网络



双向循环神经网络

I am _____
I am _____ very hungry,
I am _____ very hungry, I could eat half a pig.

I am **happy**.
I am **not** very hungry,
I am **very** very hungry, I could eat half a pig.

循环神经网络总结

- 优点：

- 引入（短期）记忆
- 图灵完备：能够近似任意可计算问题

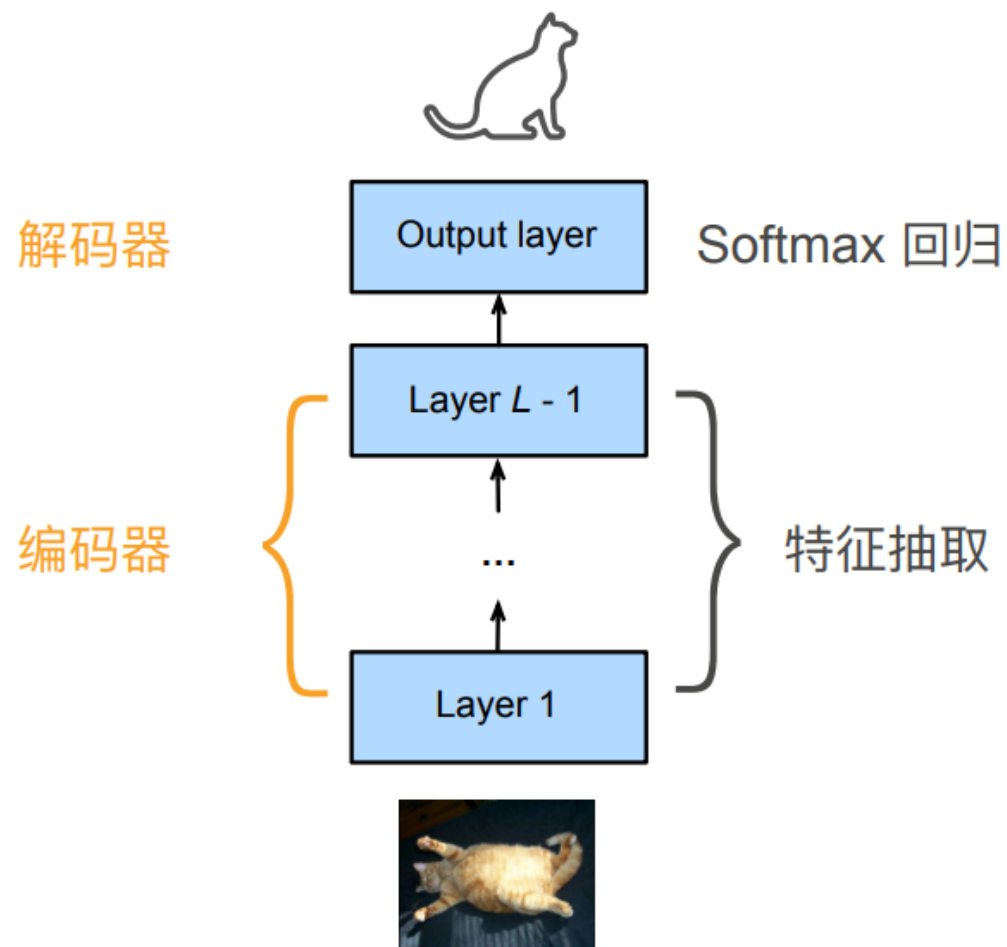
- 缺点：

- 长程依赖问题
- 记忆容量问题
- 并行能力：只能逐时间步计算

编码器-解码器架构

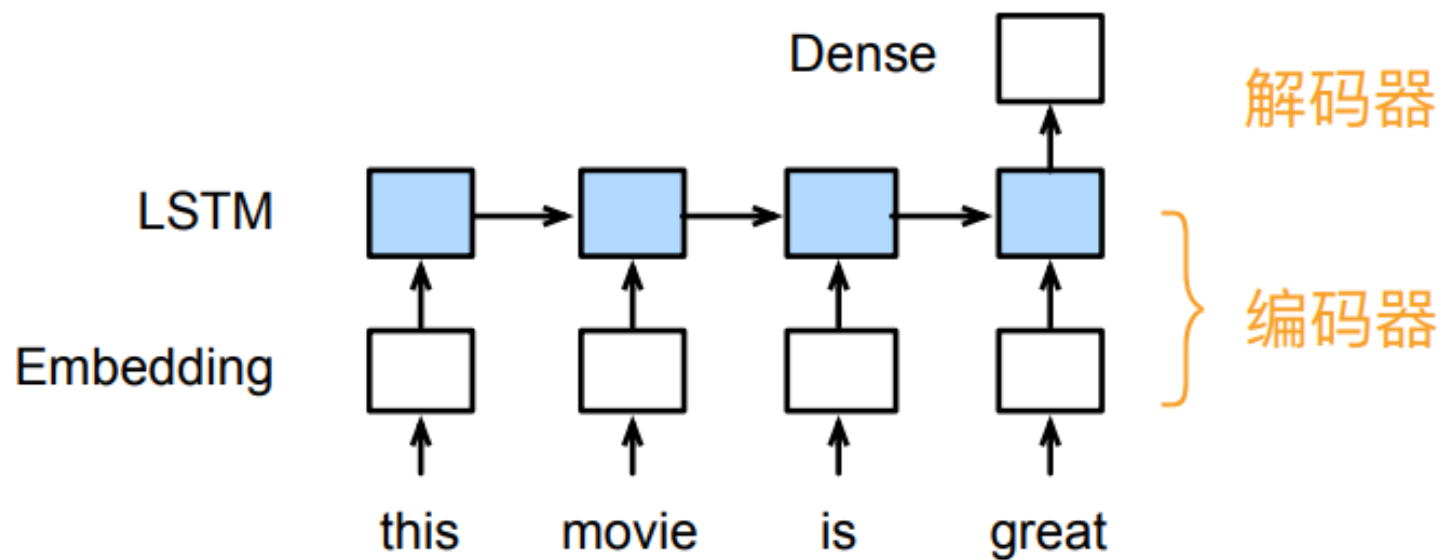


重新审视CNN



- 编码器 (Encoder) : 将输入编码为中间表达形式 (特征) ;
- 解码器 (Decoder) : 将中间表示解码成输出。

RNN中的编码器-解码器

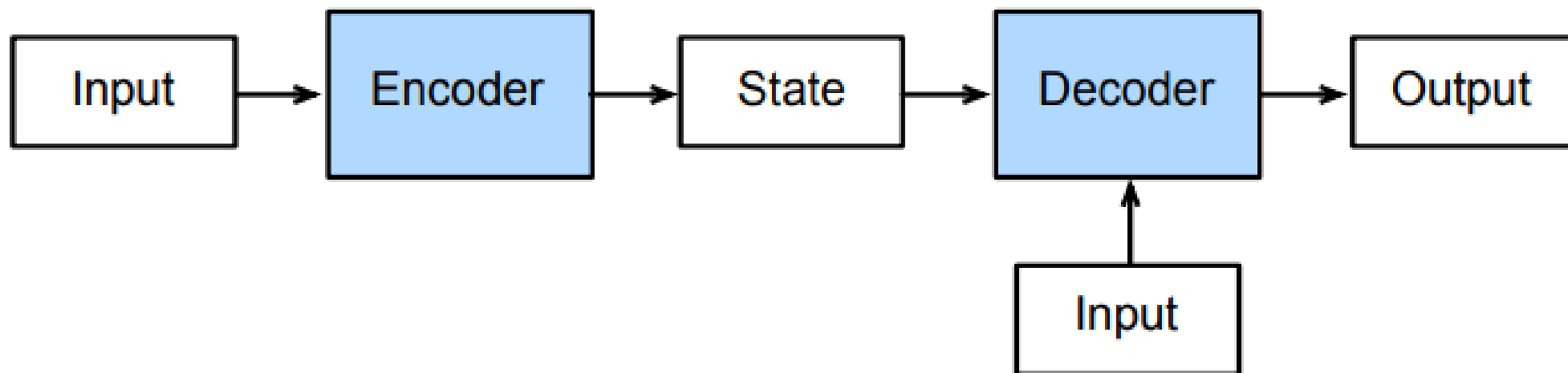


- 编码器：将序列数据表示成向量；
- 解码器：将向量表示成输出。

编码器-解码器架构

- 一个模型被分成两块：

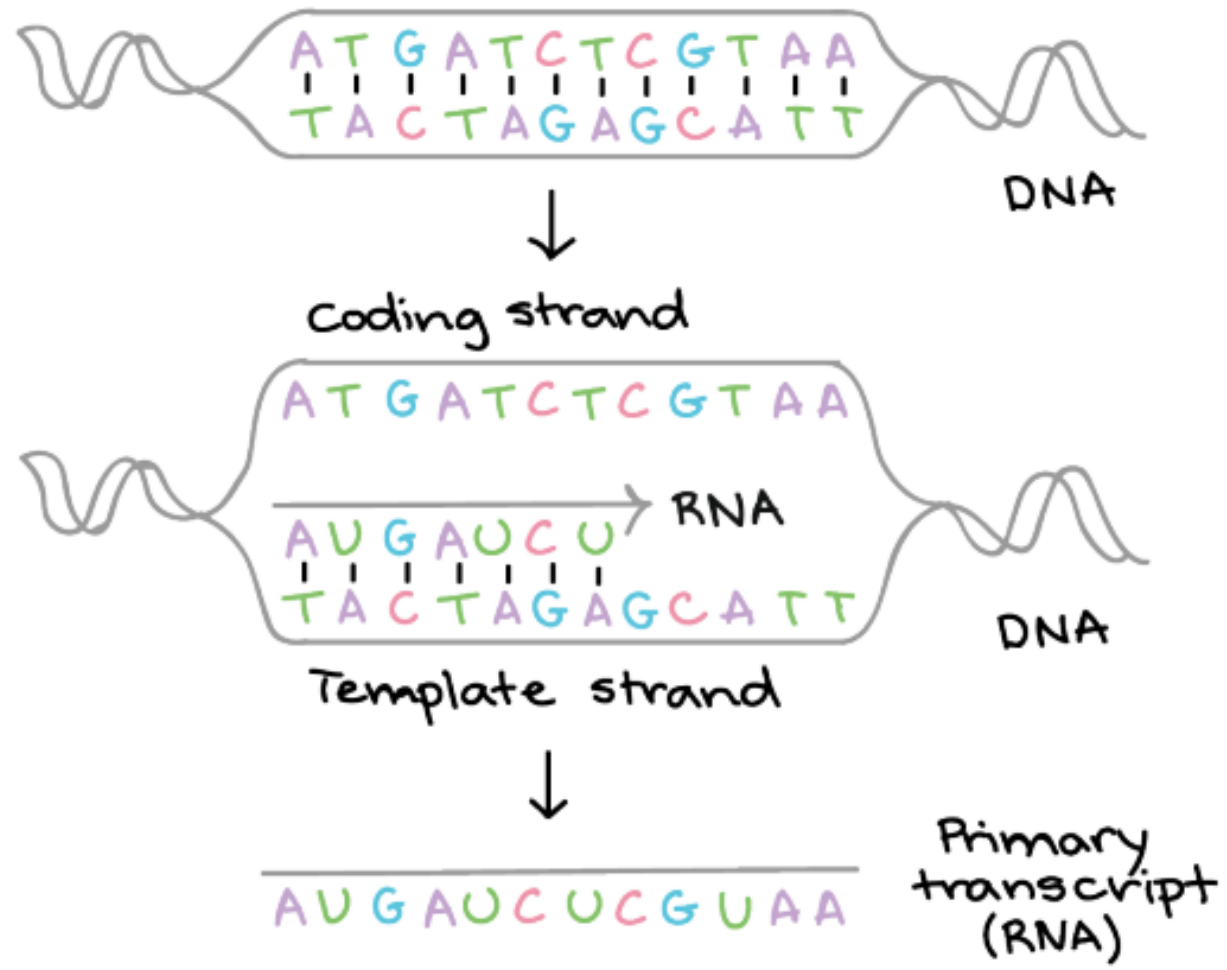
- 编码器处理输入；
- 解码器生成输出。



Seq2seq



从序列到序列



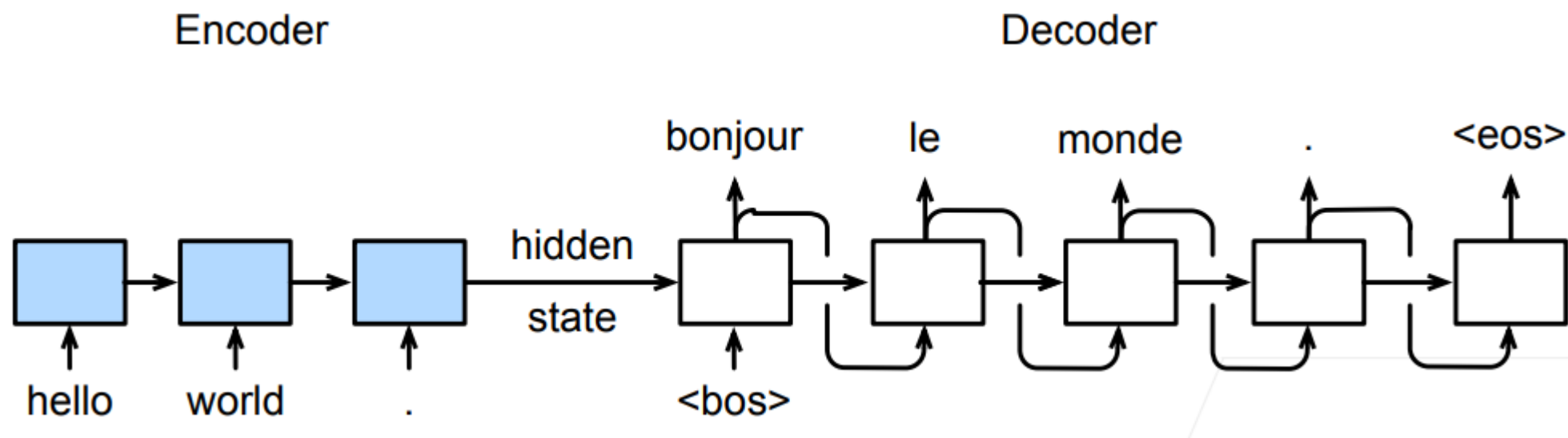
机器翻译

- 给定一个源语言的句子，自动翻译成目标语言；
- 两个句子可以有不同的长度。



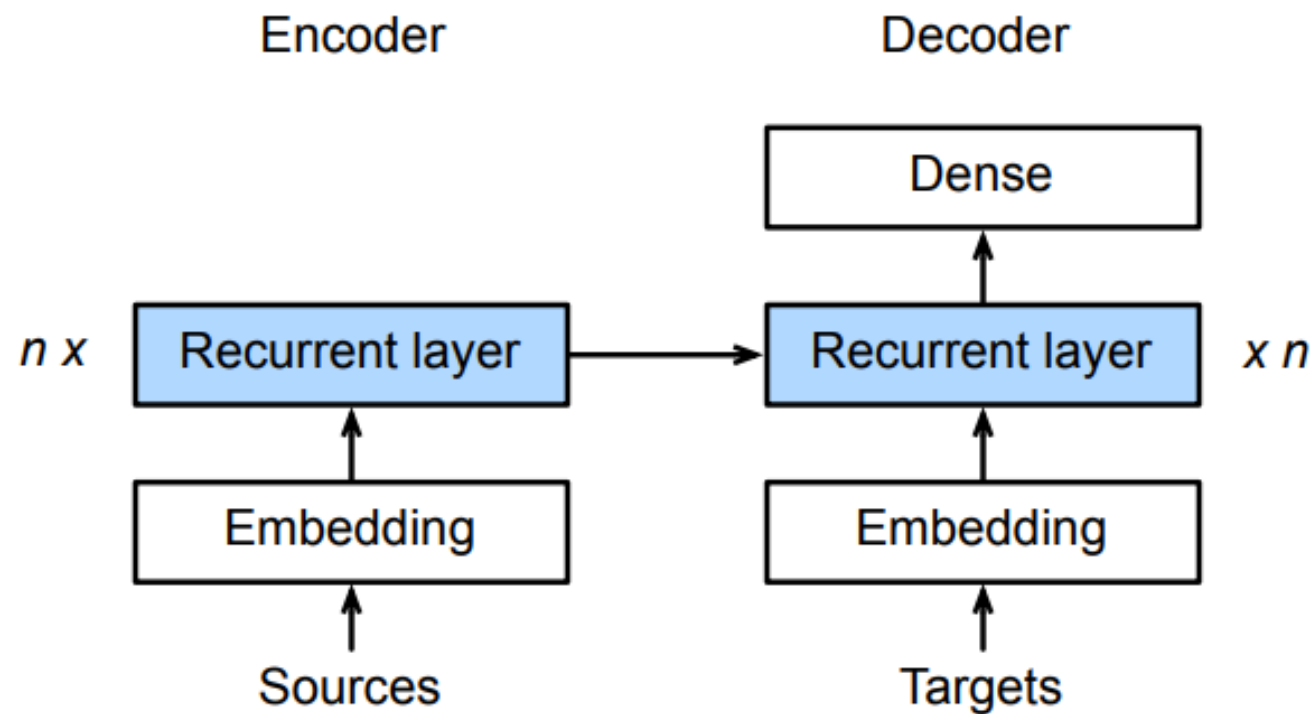
Seq2seq

- 编码器是一个RNN，读取输入的句子；
 - 此处RNN可以是双向的；
- 解码器使用另外一个RNN进行输出。

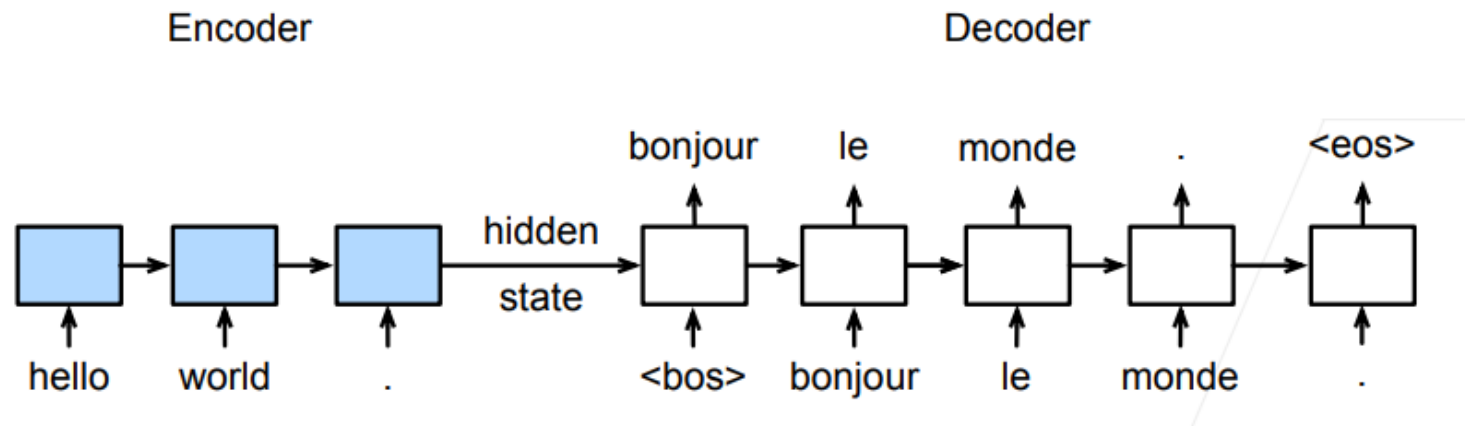


Seq2seq

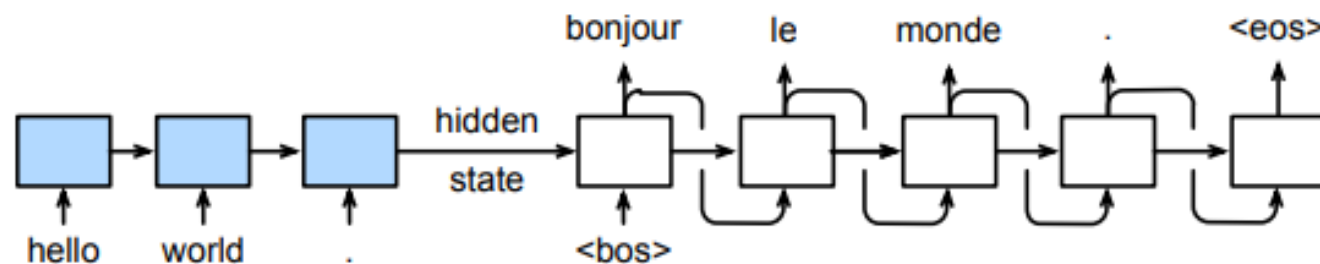
- 编码器是没有输出层的RNN;
- 编码器最后时间步的隐状态用作解码器的初始隐状态。



- 训练阶段，解码器使用目标句子作为输入

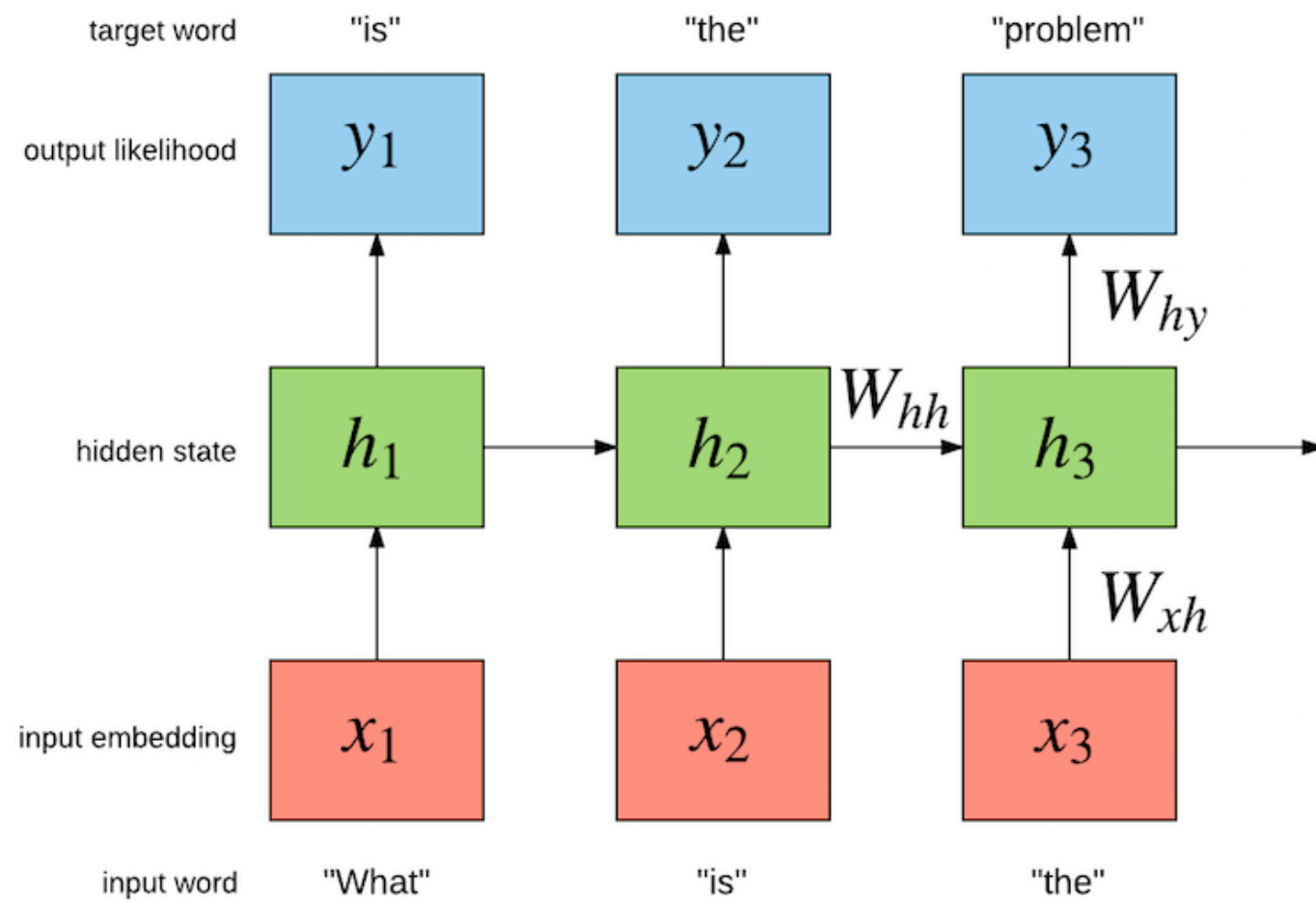


- 推理阶段，解码器无额外输入



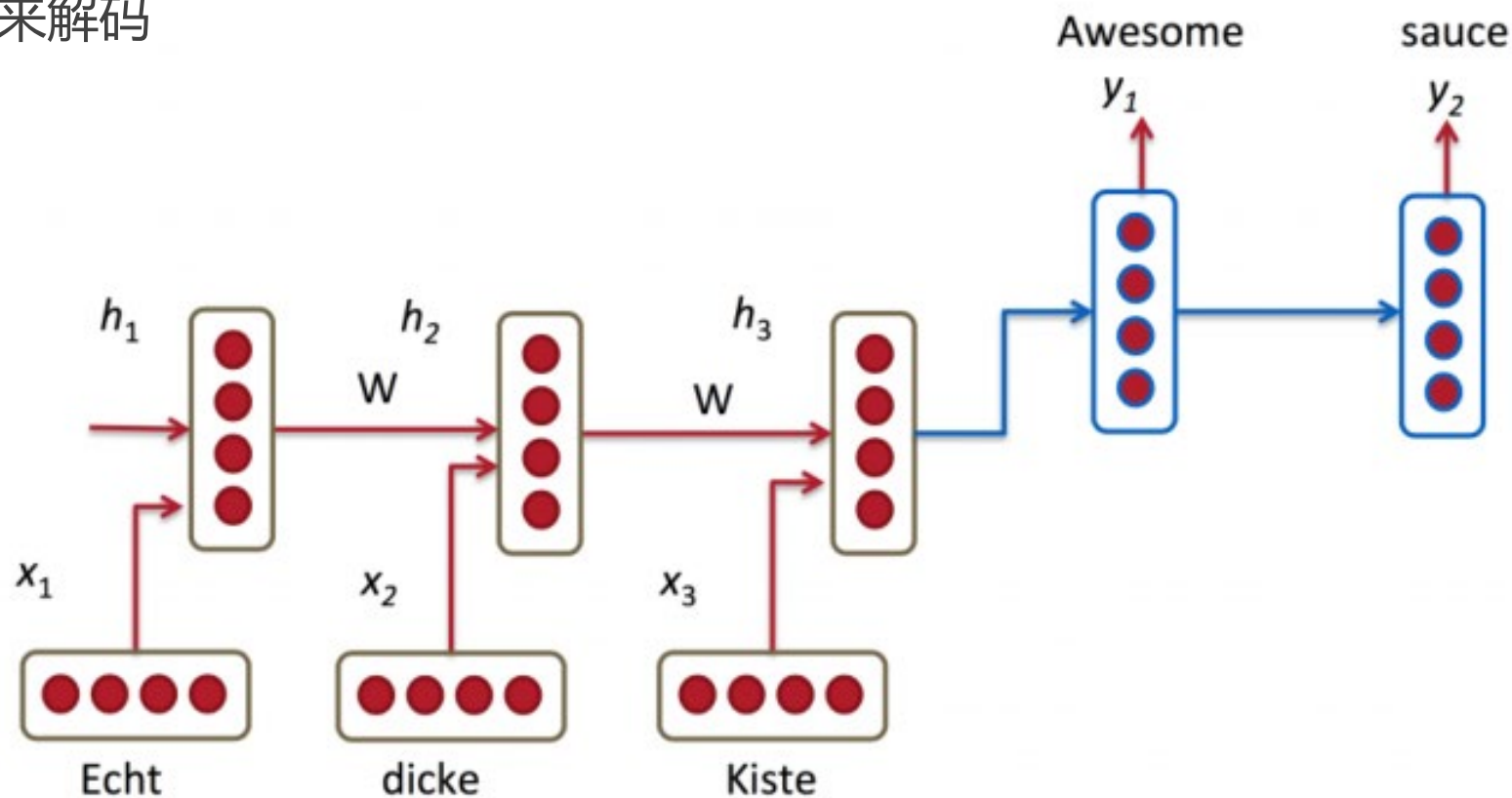
循环神经网络的应用

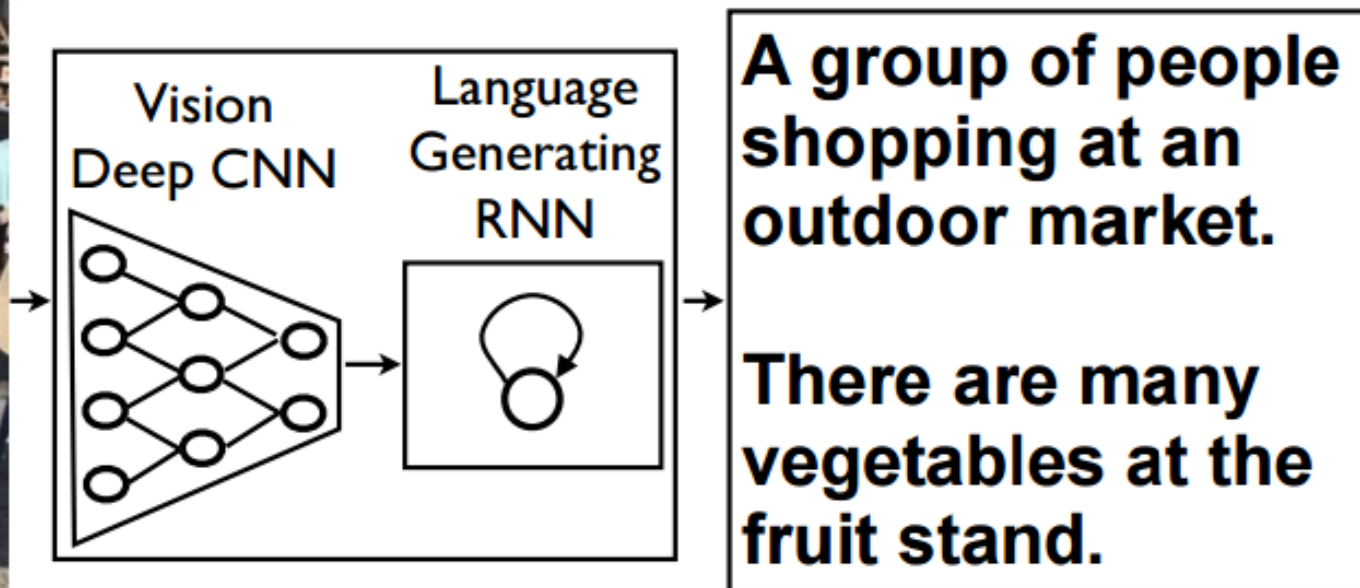




● 编码器-解码器结构

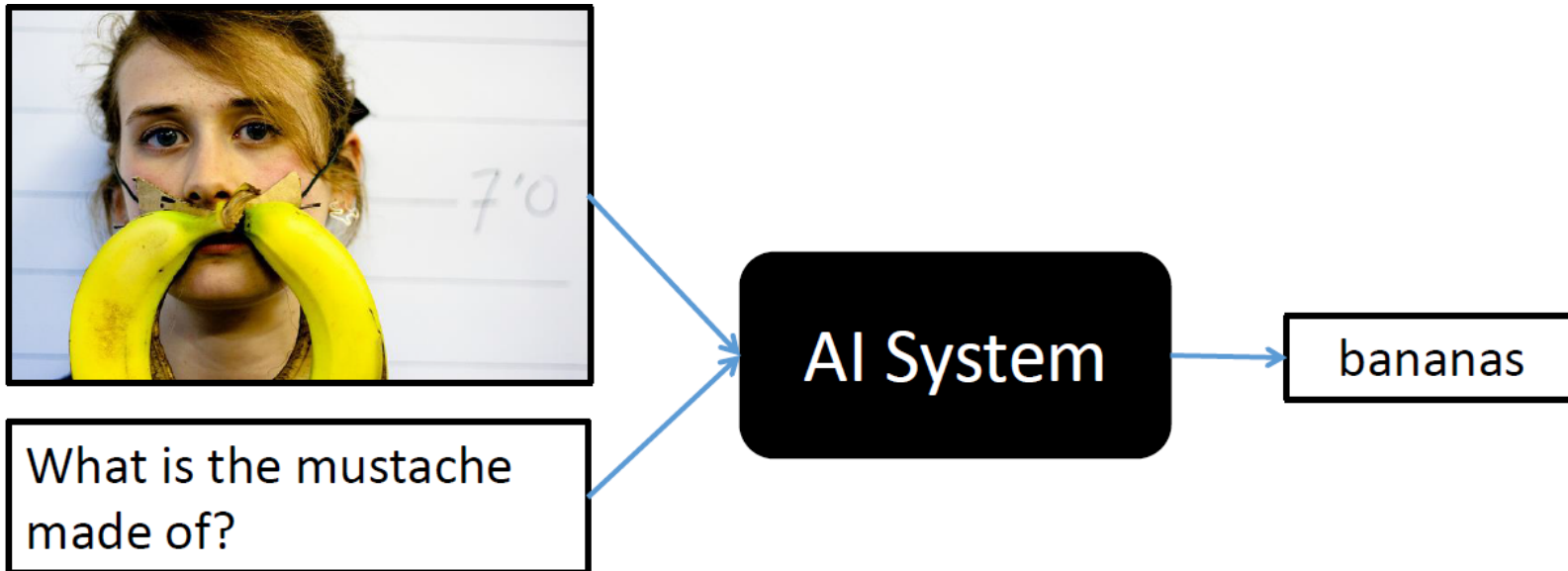
- 一个RNN用来编码
- 另一个RNN用来解码





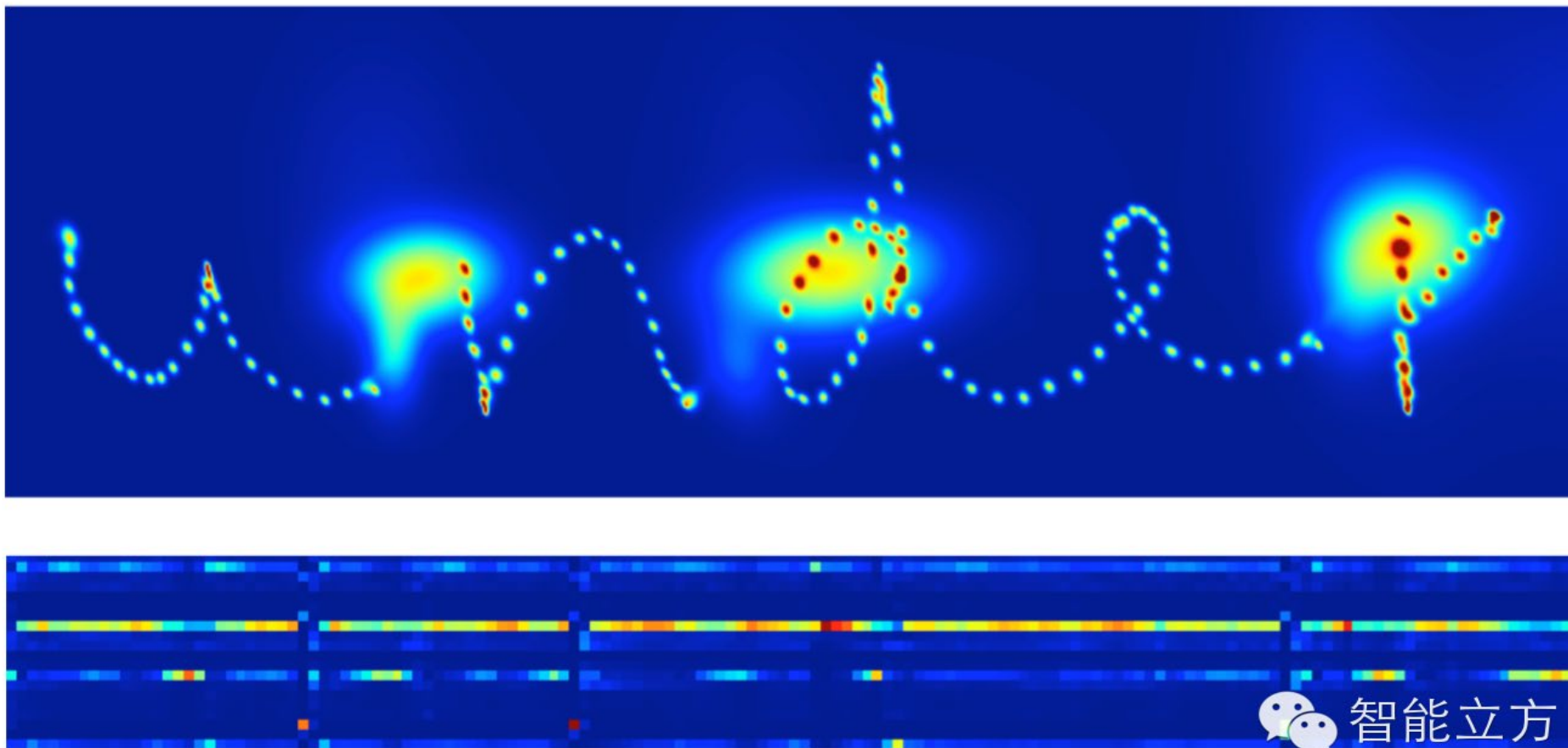
Visual Question Answering (VQA)

VQA: Given an image and a natural language question about the image, the task is to provide an accurate natural language answer

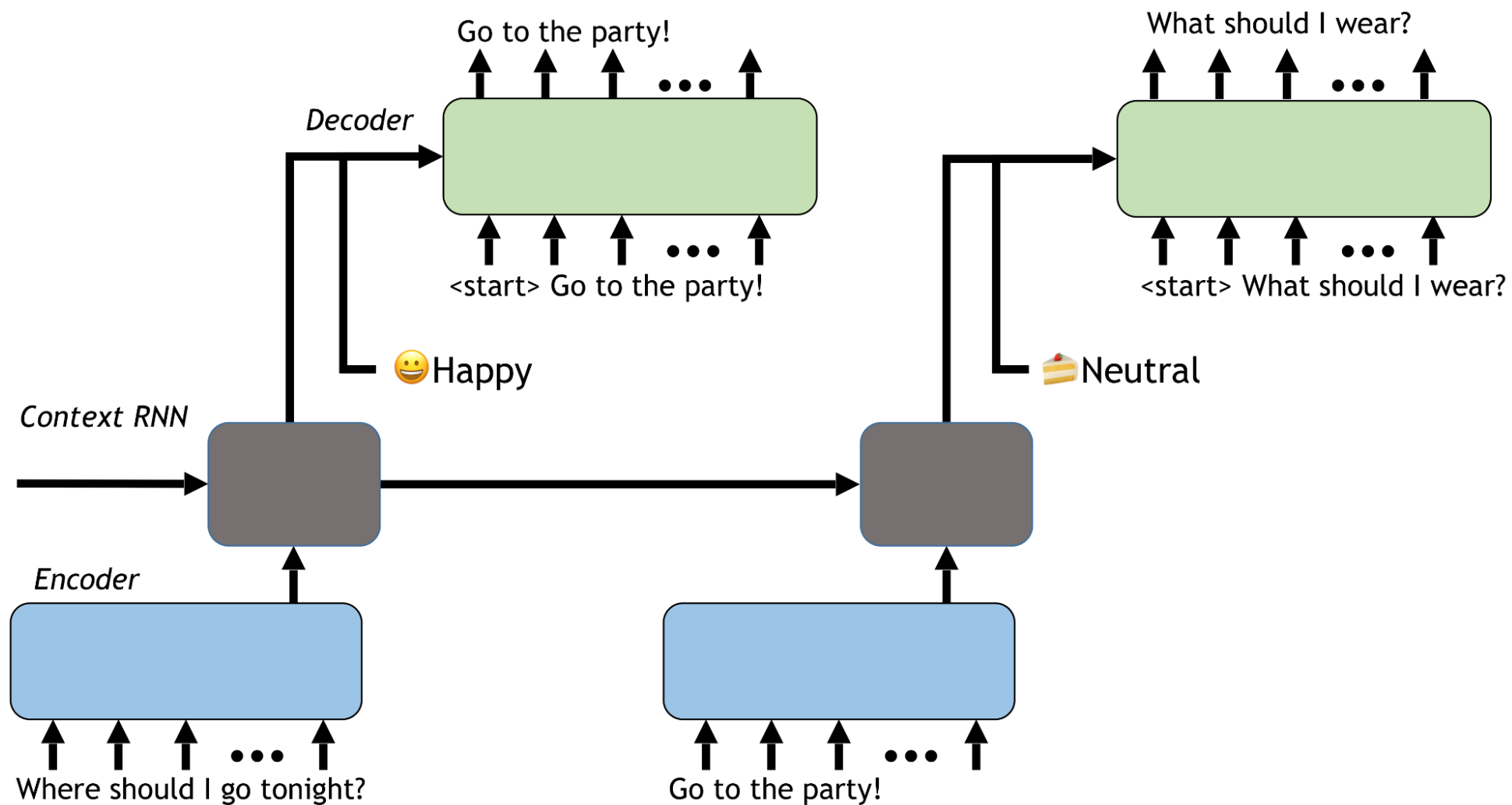


字迹模仿

- 把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为(offset x, offset y)。再增加一维取值为0或1来记录是否应该“提笔”。



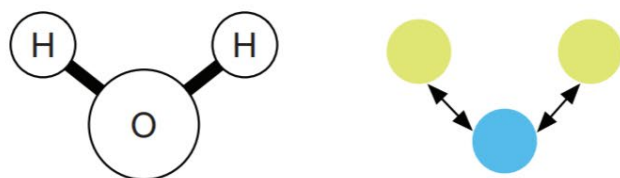
对话系统



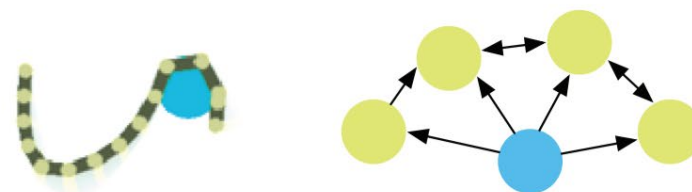
图神经网络简介



(a) Molecule



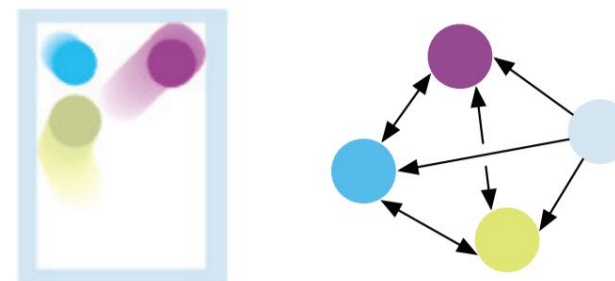
(b) Mass-Spring System



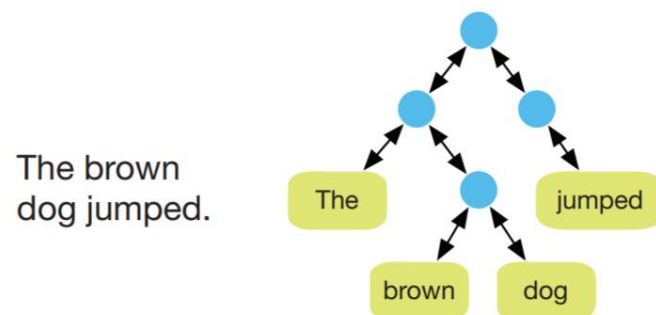
(c) n -body System



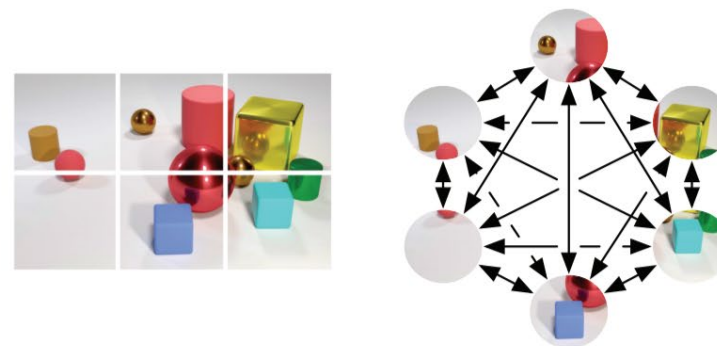
(d) Rigid Body System

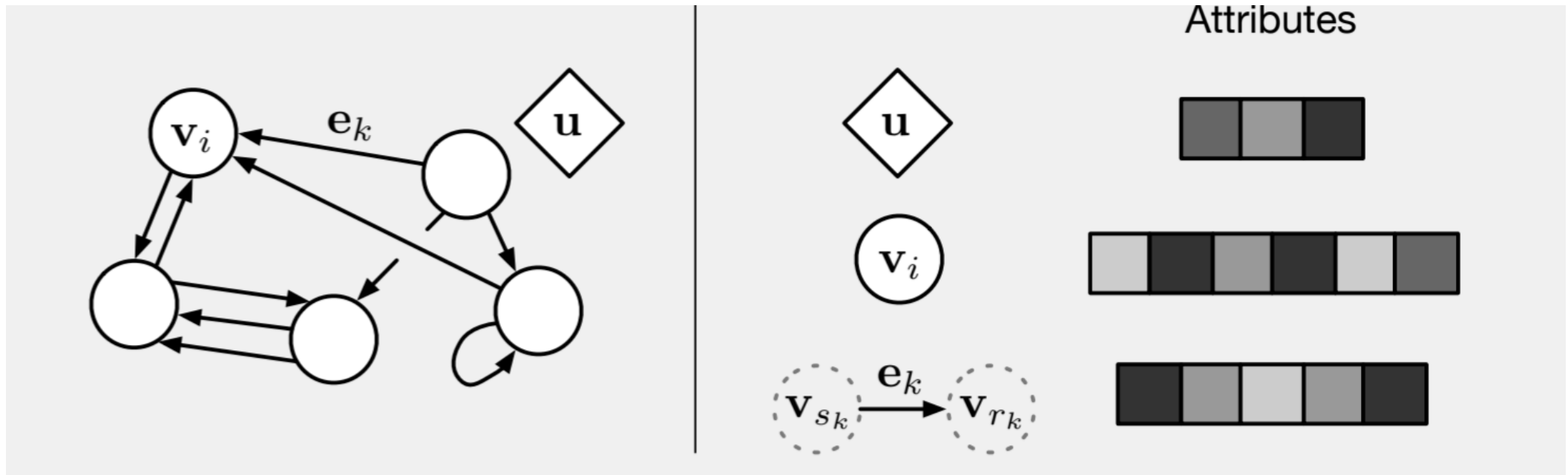


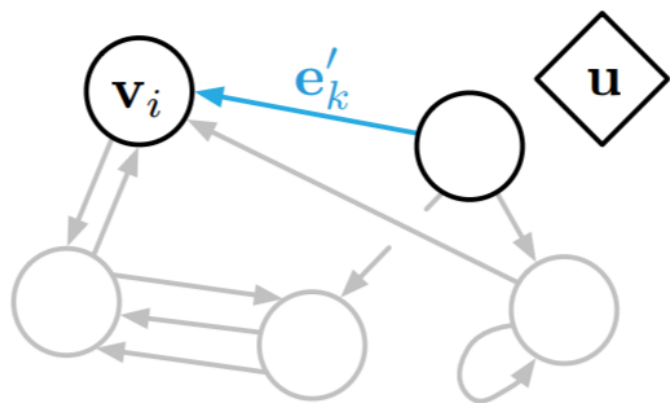
(e) Sentence and Parse Tree



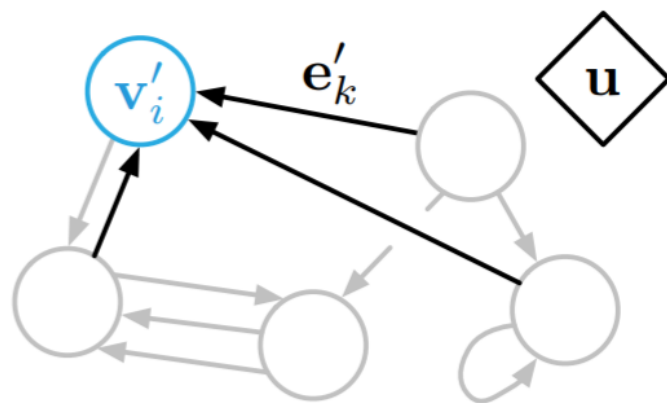
(f) Image and Fully-Connected Scene Graph



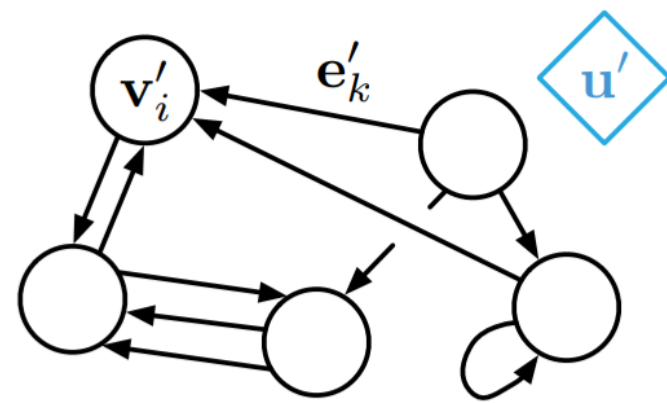




(a) Edge update



(b) Node update



(c) Global update

- 对于一个任意的图结构 $G(V,E)$

- 更新函数

$$\mathbf{m}_t^{(v)} = \sum_{u \in N(v)} f(\mathbf{h}_{t-1}^{(v)}, \mathbf{h}_{t-1}^{(u)}, \mathbf{e}^{(u,v)})$$

$$\mathbf{h}_t^{(v)} = g(\mathbf{h}_{t-1}^{(v)}, \mathbf{m}_t^{(v)})$$

- 读出函数

$$\mathbf{y}_t = g(\{\mathbf{h}_T^{(v)} | v \in \mathcal{V}\})$$

Q&A